Kayfabe v3

An architecture to argue with

Stock NVIDIA guest drivers, an emulated device, and a GPU the host keeps using.

STATUS: RESEARCH, 2026-10-27. This is a source audit and a research argument, not a new hardware qualification. The implementation baseline is immutable:

`12c4008705f17033a803e56d4d479bf1275cdb94`

That was `origin/next` when writing began. Experimental results and code outside that baseline carry separate revisions. A moving branch name never upgrades their status.

The central tension is simple: preserving a stock driver's ordering may require host-side mapping work whose latency the driver's watchdog cannot tolerate. This paper explains the boundary and gives the reader experiments that could show the explanation is wrong.

Reading route: begin with the rules, use the primer for unfamiliar terms, then read Section 6, the longest section. Section 7 asks questions that can be answered without access to this repository. Sources are indexed in Appendix C. All diagrams are conceptual TikZ drawings, not measured timelines.

Contents

1 How to attack this paper **1**

2 The problem and the product **1**

2.1 Comparisons at the width the repository supports 2

2.2 What the product argument does not establish 2

3 A primer for a programmer who knows GPUs **2**

3.1 Resource manager: names, ownership and operations 2

3.2 GPU System Processor: the peer the guest expects 3

3.3 Channels: streams that target engines 3

3.4 Timeslice groups: preserve the grouping, not just each stream 3

3.5 Unified virtual memory: presence is not fault delivery 3

3.6 VA spaces and page-directory roots 4

3.7 GPFIFO and pushbuffers: two levels of indirection 4

3.8 USERD, PUT and GET: seeing a cursor is not seeing a result 4

3.9 Doorbells: notification after publication 5

3.10 Semaphores and processed markers 5

3.11 Windows Display Driver Model 5

3.12 Timeout detection and recovery 6

4 Architecture v3: where authority and progress live **7**

4.1 Planes and traps 7

4.2 Hostile input and authored operations 7

4.3 Derive, never capture 7

4.4 Seven axes, not one compatibility version 8

4.5 Crate map and the frozen predecessor 8

4.6 Channel kinds and the meaning of completion 9

4.7 Memory store and VA manager 9

4.8 Mirrored scheduling groups 10

4.9 User-ring lifecycle: Scan, Pointer and Window 10

4.10 Lazy observation: a stale view must only delay 11

4.11 Relay, handoff and interrupts 11

4.12 Display and the broker 12

5 What the evidence establishes **12**

5.1 A result belongs to a cell and a revision 12

5.2 Correctness results and their sample units 13

5.3 A performance row with actual spread 13

5.4 The Worker GPU gate: useful, unattested and owner-controlled 13

6 The uncomfortable part **15**

6.1 The multi-application stall: start with the dependency, not its name 15
6.2 Timeline: defects that really existed, and explanations that did not survive 15
6.3 What the later walks actually say 16
6.4 How our explanation could be wrong 17
6.5 Sparse backing: semantic fidelity has a resource cost 17
6.6 Soft budgets are not deadlines 18
6.7 Mirrors: retiring memory is an ordering protocol 18
6.8 Pointer mode and lazy observation: the proof costs more than the optimisation . . 18
6.9 Relay debt and recorded disagreement 19
6.10 Poor shapes we should be willing to replace 19
6.11 Per-die constants: derive first, then admit the coverage gap 20
6.12 Security: unprivileged is a capability limit, not containment evidence 20
6.13 Fault containment, notification noise and fairness 21
6.14 Performance honesty: denominator, placement and missing samples 21
6.15 Unsupported, partially measured and explicitly postponed 21
6.16 Licensing and provenance are unfinished release work 22
6.17 Process risk: one maintainer, many agents, fallible instruments 22
6.18 Research inventory at the publication cutoff 22
6.19 A gap ledger can itself be stale 30

7 Questions for a reader without the tree **31**

A Method and reproducible commands **40**

A.1 Reconstruct the source snapshot 40
A.2 Reconstruct the PR inventory 40
A.3 Build and verify the publication 40
A.4 How a hardware result should be reproduced 41

B Glossary of terms and historical names **41**

B.1 Names encountered in historical files 42

C Evidence index **43**

C.1 Historical gap names: a coverage index, not current verdicts 47

D What changed from the predecessor **50**

1 How to attack this paper

Four labels constrain the argument. **MEASURED** means a named run or recorded observation, with a source revision and its scope. Here it always means *reported in the cited record*: this writing task used no GPU, VM or rental. **CODE READ** means a statement checked in a source file or a repository document; reading a reported result does not reproduce it. **INFERRED** means an explanation or consequence, not an observation. **UNVERIFIED** marks a question or unsupported extension; it licenses no factual assertion. An experimental proposal below is a proposed test, not a claim that its outcome occurred. These distinctions follow the repository's contributor rules. [rules]

A source key in square brackets links to an immutable repository path in the evidence index. Unless a different revision is printed there, it means the baseline on the cover. A paragraph's key scopes that paragraph; it does not turn a neighbouring inference into a measurement. Where the primary host trace is unavailable, the text says the record is a secondary report. Numbers in such a report retain that limitation even when they are precise.

CODE READ. The claims ratchet is `python3 scripts/ci/debt.py claims`. Contributors require `new=0` before publishing changed documentation and may not enrol new debt to make it pass. It is an attribution check, not a verifier of a causal explanation. In particular, its scanner covers Markdown and Rust, rather than this paper's TeX prose. The source index and editorial review therefore carry the TeX attribution obligation. A green ratchet alone does not certify this paper. [debt] [rules]

CODE READ. The predecessor records corrections to its own claims, including a runtime-init probe mistaken for a successful language-model workload, an object-graph hypothesis refuted by an already available passing control, and a mutation configuration that the tool never read. These are historical process examples, not assertions that those implementation defects remain in v3. [old]

A useful objection names a discriminating observation. For a claim about a wake, ask which waiter registered, which holder released, and whether the observer could see the event. For a claim about memory, ask who owns the backing, when the host acknowledged the change and what a failed acknowledgement leaves pending. For a speed claim, ask which revision, workload, placement, sample unit and control produced it. A count of output tokens cannot establish text correctness; an idle fault counter cannot prove that the workload reached the GPU. These are methodological requirements motivated by the predecessor's correction record. [old]

2 The problem and the product

CODE READ. Kayfabe presents an emulated NVIDIA PCI device to a guest running NVIDIA's unmodified driver. The host's NVIDIA driver stays loaded. The VMM process acts as a host driver client, using authored unprivileged operations to make the real GPU perform work. The intended benefit is simultaneous host use and guest compatibility without installing a custom forwarding driver inside the guest. This is a research implementation with no stable installation interface or public binary release. [readme] [architecture] [faq]

CODE READ. "Unprivileged" describes the host interface the design uses. It is not an end-to-end rootless certification: the public guides say the launch scripts run as root and a complete rootless run is not documented. "Stock" means no driver or kernel binary patch. Windows still requires documented registry configuration; an unmodified binary is not a zero-configuration guest. [security] [faq] [windows]

2.1 Comparisons at the width the repository supports

CODE READ. VFIO passthrough gives one guest the physical device; the host gives it up. Kayfabe instead relies on host-authored mappings and the GPU MMU. NVIDIA vGPU is a different vendor stack and licensing arrangement. The repository's positioning distinguishes API forwarding, including the sister project nvkvm-pv, which uses a custom guest module, from emulating the device for a stock guest driver. It offers no detailed Venus, virgl or native-context performance comparison, and neither does this paper. [faq]

CODE READ. The public credits describe Caraxes, an MIT fork of Leandro, as a synthetic NVIDIA GPU over vfio-user that plays the firmware role, shadows guest page tables and uses host channels while keeping the host NVIDIA driver. They describe Open-IOV's goal and host-module approach, but explicitly do not establish whether its host keeps using the GPU. These are repository-attributed descriptions, not an independent audit of either project. The credits identify nvproxy as a source of capability allowlists, not as a claim that it implements this stock-device architecture. No claim about relative speed, security or unpublished implementation details follows. [credits]

2.2 What the product argument does not establish

CODE READ. A hostile-guest model is a design target, not an independent security audit. Multiple VMs on one GPU, shared-fault containment and resource fairness need evidence of their own. A passing Linux thin guest is not a passing Windows desktop; a passing die does not stand for an entire family; a fast isolated renderer does not establish multi-application stability. The public FAQ explicitly separates these cases. [security] [faq]

INFERRED. The compatibility surface is a moving dependency on the guest driver's observable expectations. This is valuable when it works and costly when a seemingly unimportant ordering or missing response sends the driver down a different path. The seven compatibility axes and the failure case below are ways to make that cost inspectable, not a promise to eliminate it.

3 A primer for a programmer who knows GPUs

The vocabulary here is deliberately limited to what the architecture needs. These are operational descriptions from this repository's architecture, channel design and guest guides, not a complete specification of NVIDIA hardware. [architecture] [kinds] [windows]

3.1 Resource manager: names, ownership and operations

CODE READ. RM means *resource manager*. A client requests memory objects, address spaces, channel groups and channels, then refers to them by handles. Kayfabe models the guest-side object relationships and separately authors host-side requests. A guest handle must never become host authority just because both are integers. The host RM remains trusted to enforce its own memory and channel rules. [architecture] [rmcode] [security]

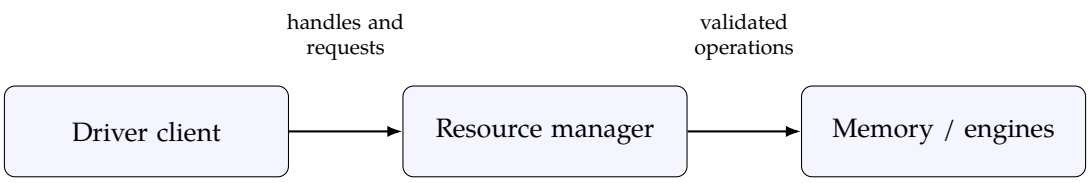


Figure 1: A resource manager names and controls resources; a handle is not a bus address.

3.2 GPU System Processor: the peer the guest expects

CODE READ. GSP means *GPU System Processor*; its resource-manager firmware is a peer of the CPU-side driver. The guest submits framed requests in message queues and expects replies and notifications. Kayfabe plays that peer, including boot and queue state, rather than running vendor firmware in the emulated device. An RPC is a remote-procedure-call message; accepting a request frame and executing its meaning are separate obligations. [gspcode] [architecture]

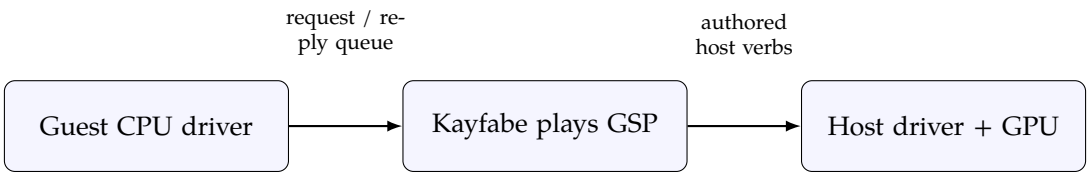


Figure 2: The firmware-facing protocol is emulated; no vendor firmware runs in the emulated device.

3.3 Channels: streams that target engines

CODE READ. A channel is a submission context with ordered work and state. A copy engine moves or fills memory; graphics, compute and video engines do different work. A channel's kernel privilege and address space determine what it can reach. The word "user" here describes GPU execution privilege, not whether the host launcher happened to be root. Kayfabe distinguishes user work, translated kernel work and work emulated entirely by the VMM. [kinds] [security]

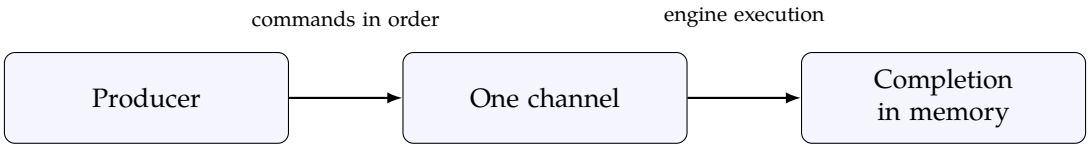


Figure 3: A channel is an ordered submission context, not a CPU thread.

3.4 Timeslice groups: preserve the grouping, not just each stream

CODE READ. TSG means *timeslice group*, a scheduling group of channels. Kayfabe's declared mapping is one host TSG for each guest TSG. Creating every host channel independently could preserve individual command bytes while changing the grouping the driver expects. Scheduling metadata and channel membership therefore belong in the model as well as commands. [kinds] [architecture] [tsgcode]

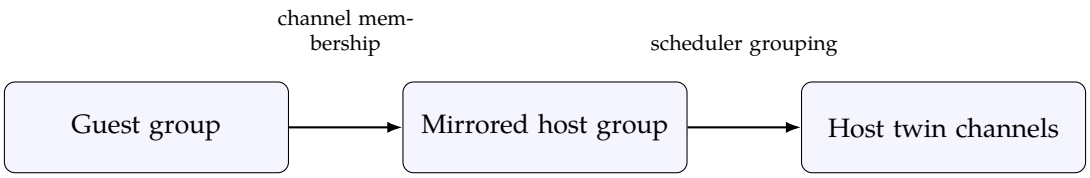


Figure 4: A timeslice group keeps related channels together at the host scheduling boundary.

3.5 Unified virtual memory: presence is not fault delivery

CODE READ. UVM means *unified virtual memory*. Managed allocations may need a page supplied when an access first occurs, then the access replayed. Mirroring mappings that already

exist solves a different problem. The repository's remaining Linux application failures involve this demand-paging path; an optional host-side research route is distinct from the stock product path. [status] [uvmdesign] [faq]

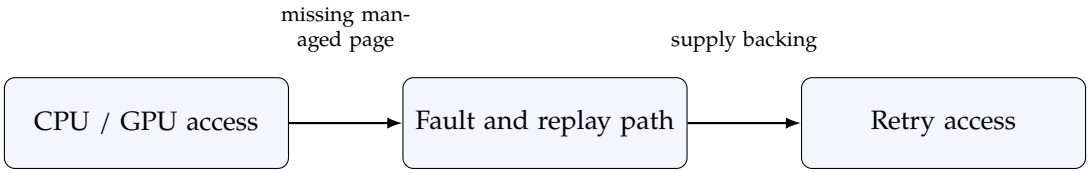


Figure 5: Demand paging requires a fault path, beyond copying already present mappings.

3.6 VA spaces and page-directory roots

CODE READ. A virtual-address (VA) space gives virtual addresses a meaning. A page-directory base, or PDB, names the root used for a particular GPU's page tables. The space is an object; the root is an attribute that can change or be absent. Kayfabe mirrors the guest's mappings with its own host requests. It does not tell the host GPU to adopt hostile guest page tables. That distinction makes acknowledgement and root lifetime central. [architecture] [memcode]

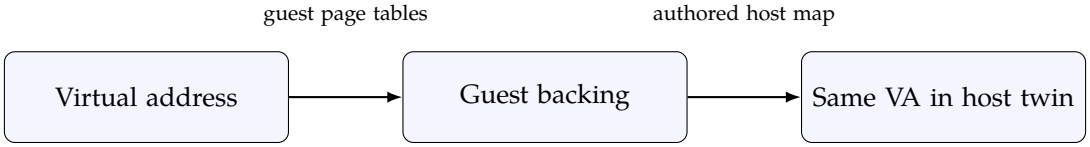


Figure 6: Mirroring preserves virtual names while the host owns the actual page tables.

3.7 GPFIFO and pushbuffers: two levels of indirection

CODE READ. GPFIFO is the GPU's command-reference ring. A ring entry identifies a pushbuffer, which contains method packets and arguments. A method is a command to a bound engine object; parsing it requires packet state across entries. Passing an entry's pointer to hardware and copying or interpreting the pointed-to bytes are therefore different operations. A parser that assumes every entry starts at a packet boundary can be wrong even when every pointer is in bounds. [relaydesign] [lifecycle] [turncode]

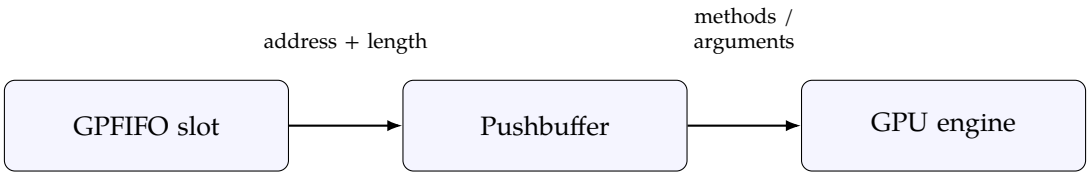


Figure 7: A command ring contains references to command buffers; it is not the command bytes.

3.8 USERD, PUT and GET: seeing a cursor is not seeing a result

CODE READ. USERD is memory holding channel producer/consumer state. PUT advertises work; GET records fetching progress. Kayfabe also uses a processed reference marker when it needs proof that commands crossed a particular point. The event-loop investigation found that GET could move beyond an entry while execution behind a dependency had not occurred. It cannot serve as a general execution-completion fence. [eventloop] [lifecycle]

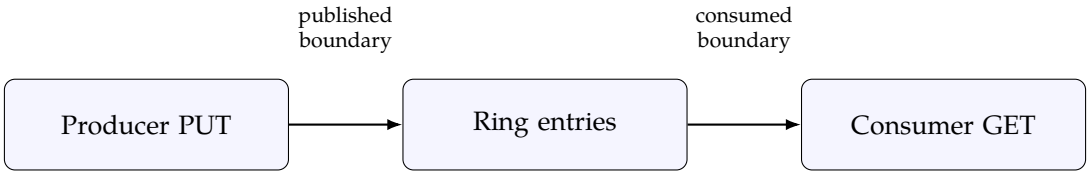


Figure 8: USERD carries producer and consumer state; fetching is not finishing.

3.9 Doorbells: notification after publication

CODE READ. A doorbell is a register write telling the consumer to inspect published work. A token selects an admitted channel route. Guest userspace can write tokens; the trap cannot infer a trustworthy process identity from that write. The contract requires discovery of work covered by a doorbell, including races with a servicing worker. It does not allow a timer poll to stand in for a missing wake. [architecture] [rules] [trapcode]

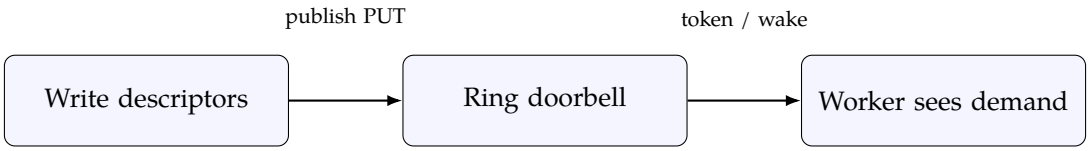


Figure 9: A notification follows publication; it does not transfer ownership of arbitrary bytes.

3.10 Semaphores and processed markers

CODE READ. A semaphore is a memory location used to express a dependency: earlier work releases a value and later work acquires it. A non-stall interrupt is an asynchronous notification that invites software to check its own completion state. A SYNC_WAIT barrier constrains when later command bytes may be consumed. These are related mechanisms, not interchangeable proofs. The relay's processed-reference marker exists because a fetch cursor was too weak a witness. [eventloop] [kinds] [faq]

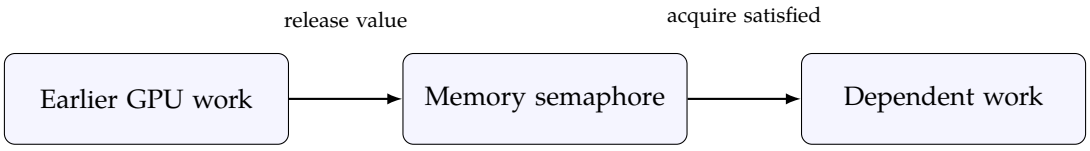


Figure 10: A dependency value is distinct from a CPU interrupt and from a fetch cursor.

3.11 Windows Display Driver Model

CODE READ. WDDM means *Windows Display Driver Model*. For this argument, the important part is that Windows schedules GPU work, observes completions and can initiate recovery. A Windows scheduling packet is not automatically identical to one private host-ring publication. Establishing that correspondence is an instrumentation problem, not a naming convention. Stock Windows and Linux drivers can exercise different device protocols. [windows] [latency] [faq]

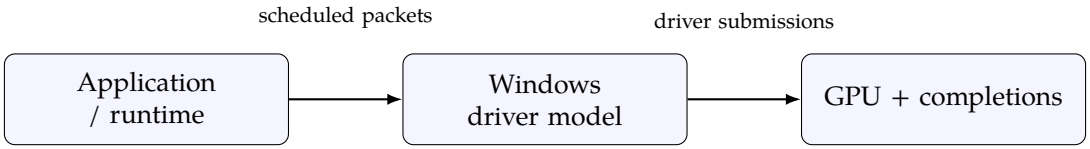


Figure 11: The Windows display-driver model adds scheduling and recovery around device work.

3.12 Timeout detection and recovery

CODE READ. TDR means *timeout detection and recovery*: Windows can decide that GPU progress took too long and recover the driver. A changed diagnostic timeout can distinguish late work from work that never completes; it does not make the production ordering acceptable. The project forbids treating a changed timeout as a product fix. A completion observed after recovery also needs a lifetime identity before it can be attributed to the original work. [rules] [walknarrow] [latency]

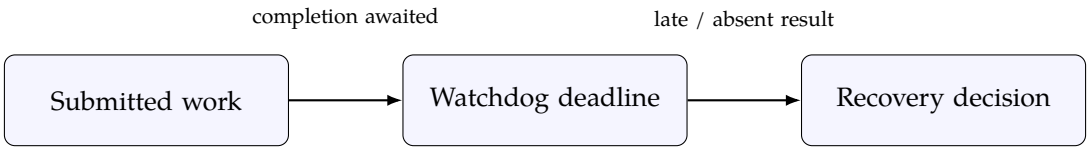


Figure 12: Timeout detection and recovery observes elapsed progress, not our preferred explanation.

4 Architecture v3: where authority and progress live

4.1 Planes and traps

CODE READ. The product is the `kf-*` tree linked into the VMM process. A BAR0 write records demand through bounded token and ring state. Workers service channel demand; a register drainer services privileged register operations. BAR1 and BAR2 memory accesses do not trap. There is no separate isolate process or product IPC plane. Confinement applies to the whole hypervisor process. [architecture] [corecode] [trapcode]

Invariant: No host mapping or blocking operation runs on a vCPU or under a lock it takes.

Test: The trap crate's unit tests, core composition tests and source restrictions constrain the reachable trap path. Hardware trap timing checks a separate latency property. [trapcode]

What the test could miss: A nonblocking-looking call can acquire a lock held by slow host work. A mock host that never blocks cannot demonstrate the production lock discipline. The old paper records that failure mode; the restriction must be checked transitively. [old]

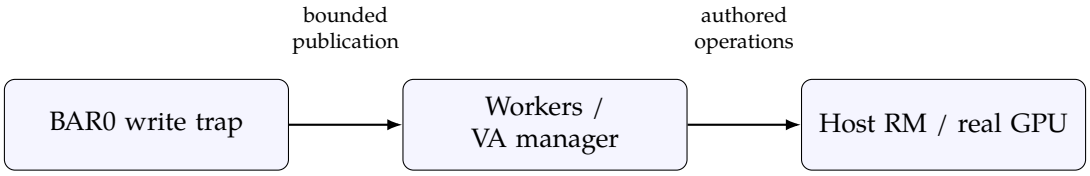


Figure 13: The vCPU records demand; host work belongs to other threads.

4.2 Hostile input and authored operations

CODE READ. Guest root and guest userspace are untrusted. Host memory operations are constructed from admitted ownership, extent, permission and kind information; guest control messages are not replayed to the host. Host GPU facts come from unprivileged queries or generated open interfaces. This is the intended authority boundary, with the host driver and GPU trusted. [security] [architecture] [hostcode]

Invariant: No guest-chosen raw address or privilege request creates new host authority.

Test: Hostile range/ownership tests, channel privilege tests and the physical-operand hardware oracle exercise different pieces of this claim. [security] [physevidence]

What the test could miss: Checking a decoder cannot establish that every caller uses it. A guest-declared OS identity also cannot prove a safe mapping scope. Whole-path reachability and multiple-VM fault effects remain separate review obligations. [rulings] [security]

4.3 Derive, never capture

CODE READ. The architecture prefers facts read from the host's own unprivileged interface, computed virtual-device state and rules generated from pinned open headers. A device identity is not permission to copy another die's answer. Request validation and field-level virtual support still apply when an answer comes from the host. Published measured exceptions belong to exact die/driver coverage, not an invisible default table. [architecture] [constants]

Invariant: Every guest-visible field has a source and a defined unsupported case; an unknown die never silently inherits a neighbour's observation.

Test: The constants strategy requires source tags, derivation oracles, wrong-answer mutations and

same-die native acceptance. It explicitly separates the implemented baseline from table consumers on branches. [constants] [provenance]

What the test could miss: A source tag proves provenance only if its source actually supports the field. A boot that never exercises an optional feature cannot verify that feature, and agreement between a few dies is not a mechanism for a family rule. [constants]

4.4 Seven axes, not one compatibility version

CODE READ. The design separates guest kernel, guest driver, host driver, GPU architecture, GPU die, VMM and guest OS. An architecture supplies family mechanisms; a die supplies actual inventory and capability facts. The guest ABI controls decoding; the host ABI controls authored calls. The VMM supplies memory, interrupts and lifecycle events. These are distinct axes even when a test holds several fixed. [design] [constants] [driverdesign]

Axis	Boundary to inspect	Counterexample worth testing
Guest kernel	DMA addressing / virtual IOMMU	IOVA mistaken for guest physical address
Guest driver	versioned decode / accepted replies	layout changes within a branch
Host driver	authored RM ABI	successful syscall with failed in-band status
Architecture	class / register / MMU family	different engine or boot protocol
Die	runtime facts / measured exceptions	floor-swept or unmeasured die
VMM	memory, IRQ, reset adapter	reset origin or wake semantics differ
Guest OS	driver protocol and scheduling	Windows registration / recovery path

The counterexamples are proposed attacks, not claims that each has occurred at the baseline. A matrix that changes one axis at a time is evidence for those cells, not the full Cartesian product. [driverdesign] [constants]

Invariant: Guest and host ABI choices remain independent, and OS or VMM identity never grants extra memory authority.

Test: The driver matrix tests named host/guest cells; versioned decoder oracles check layouts. Portability beyond QEMU and x86-64 still needs real adapter evidence. [driverdesign] [requirements]

What the test could miss: A layout may change within a driver branch, and a cross-build cannot exercise mapping, interrupt or reset semantics. Passing one axis with the others fixed is not the full grid.

4.5 Crate map and the frozen predecessor

CODE READ. The following is a functional map from `ARCHITECTURE.md`, not a generated claim about every dependency edge. [architecture]

Crates	Responsibility
`kf-trap`, `kf-core`	bounded demand publication and plane composition
`kf-gsp`, `kf-rm`	firmware-facing protocol and resource-manager model
`kf-chip`, `kf-abi`	host facts, family rules and versioned generated layouts
`kf-host`	one in-process host RM session and authored verbs
`kf-mem`, `kf-cuda`	backing store, VA manager, GPU walk and diff protocol
`kf-chan`	channel birth, translated work, relay and completions
`kf-qemu` + C device	QEMU memory, lifecycle, interrupt and device adapter
`kf-disp`, `kf-broker`	emulated display and VMM-neutral broker protocol
`kf-oprom`, `kf-gop-image`	generated boot-display container and build
`kf-harness`	real-GPU tests without QEMU
`kf-census`, `kf-crec`, `kf-trace`	offline tools and historical oracles
`kf-arch`, `kf-linux-raw`, `kf-util`	vocabulary, audited OS calls, utilities

CODE READ. The older `kayfabe-*` crates remain frozen; the raw-client grader depends on part of that tree. Product `kf-*` crates do not depend on it. Unsafe code is confined to named adapter crates and `*_unsafe.rs` files. A crate inventory is a source-level fact; it says nothing about whether a feature has a production caller. [rules] [architecture]

Invariant: Product crates retain their dependency and unsafe boundaries; the frozen grader tree never becomes a hidden product dependency.

Test: Workspace manifests and CI boundary checks enforce structural restrictions. Reached-path oracles and real adapter tests address behaviour separately. [rules] [architecture]

What the test could miss: A legal dependency graph can still contain an unused implementation or a production-only blocking path. Source structure does not establish reachability or hardware correctness.

4.6 Channel kinds and the meaning of completion

CODE READ. Passthrough creates a host twin for user work and lets the real engine execute its commands. Translated channels rewrite kernel copy work into an owned address window and submit it to the real engine. Emulated channels execute only work whose meaning belongs entirely to the VMM. The three kinds differ in authority and completion, not simply speed. [kinds]

Invariant: No CPU executor substitutes for GPU work; no completion is forged for work submitted to hardware.

Test: The channel suites and real-engine gates check actual execution, content and notification. The translated-plane design separates address arithmetic from execution. [translated]

What the test could miss: An application can report success without checking its GPU error or result. A harness must validate the output and completion path, not merely the process exit. [status]

4.7 Memory store and VA manager

CODE READ. Guest video memory is one host allocation with offsets corresponding to guest framebuffer offsets. Guest system memory comes from the VMM's shared backing. A GPU walk reads guest tables and diffs them against confirmed host placements. Host maps/unmaps apply the changes, invalidate, and acknowledge runs. A refused map remains a difference. The committed-placement record is not an adopted guest page table. [architecture] [diffcode] [memcode]

Invariant: Only acknowledged host placements become committed state. Old backing remains owned until the relevant invalidation acknowledges its retirement.

Test: The diff model, apply tests and hardware walk gate compare emitted runs, acknowledgements and resulting translations. The retirement experiment adds explicit delayed-free tests. [diffcode] [applycode] [retirementpr]

What the test could miss: A software model can assume host RM ordering that hardware does not provide. Table roots, shared subtrees, root withdrawal and failed host invalidation require separate schedules. Whole-space scanning is not equivalent to remapping every unchanged leaf. [latency]

4.8 Mirrored scheduling groups

CODE READ. A host TSG mirrors a guest TSG, and twin lifetime follows the trusted object model. The late-joiner design documents why a newly added channel cannot silently miss earlier scheduling state. Address-space mirroring and TSG mirroring are separate concerns: preserving one does not prove the other. [tsgcode] [latejoin] [kinds]

Invariant: Channels in one guest scheduling group retain the corresponding host grouping and required state as members arrive and leave.

Test: Grouping tests and the video-engine validation exercise group creation and related channel use. [tsgcode] [video]

What the test could miss: A test with all channels present at initial configuration misses late arrival. Reused handles and reset generations can make a correct group lookup refer to the wrong life. [latejoin]

4.9 User-ring lifecycle: Scan, Pointer and Window

CODE READ. Windows user-work can require software intervention around registration and ring migration. In Scan mode the worker inspects command packets. Pointer mode forwards admitted GP references without reading the pushbuffer. Window mode restores scanning across the registration boundary. The clean anchor must have been observed before the registration reply; a later convenient cursor cannot retroactively establish that boundary. [lifecycle]

Invariant: Doorbell-covered entries appear once, in order and bit-exact; Pointer reads no pushbuffer; a validated descriptor is the snapshot actually published.

Test: Real-turn model, property and threaded stress suites cover mode transitions, ring migration, packet continuation, poisoned bytes above PUT and registration at different positions. [lifecycle] [turncode]

What the test could miss: Finite model schedules can omit a real host lock holder, DMA ordering or guest mutation. A normal-user experiment cannot certify a hostile user that rewrites its own memory. The boundary remains native USER privilege plus admitted mappings. [rulings]

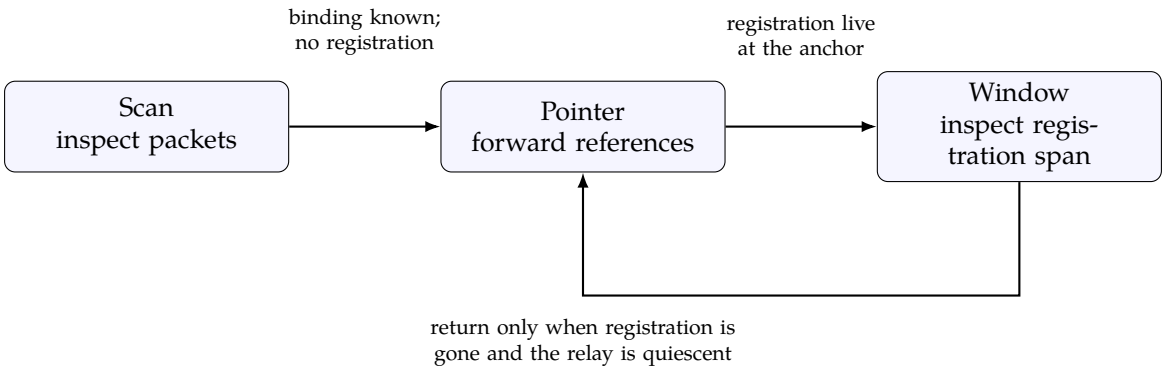


Figure 14: Main lifecycle path; direct Scan-to-Window and teardown edges are omitted. Registration and packet-boundary rules govern each transition. [lifecycle]

4.10 Lazy observation: a stale view must only delay

CODE READ. Pointer mode caches a previously loaded PUT within the same ring generation and reduces private USERD observations. Scan and Window remain fresh. Low credits, waits, split segments and a full guest ring force observations. A stale GET may reduce available credit; it must never increase credit or release a marker gate early. Mode changes and migration invalidate cached state. [lifecycle] [lazycode]

Invariant: No idle decision relies on cached PUT, and no stale observation is the sole missing wake for a wait that can already be satisfied.

Test: The lazy-turn suites compare forwarding identity with observation enabled and disabled, exercise small rings, and test forced observations and generation changes. [lifecycle]

What the test could miss: A counter of fewer reads is not proof of liveness. Combined performance arms cannot attribute speed to each optimisation separately. Hardware completion and holder-release timing remain outside the pure predicate model. [lifecycle]

4.11 Relay, handoff and interrupts

CODE READ. The relay has explicit progress obligations. If a storage, row or page-table holder prevents progress, release must arrange another attempt; repeatedly trying a busy lock is not an event-delivery proof. The baseline includes the relay handoff fix cited by the README and a later merged lost-wake correction. Experimental split-latency instrumentation keeps that relay design rather than introducing an attempt limit. [readme] [turncode] [latency]

Invariant: Registering a waiter concurrently with completion or lock release cannot lose the last wake. An interrupt invites a check; it does not manufacture a completed fence.

Test: Relay interleaving tests, completion tests and the live regressions cited in the README exercise parts of this claim. The later probe experiment explicitly diagnoses missing wakes. [readme] [probepr]

What the test could miss: Host events are GPU-wide, so an interrupt can be unrelated to this channel. Observing a marker late does not timestamp when the GPU wrote it. A worker busy elsewhere can delay observation without a lost physical event. [faq] [latency]

4.12 Display and the broker

CODE READ. An emulated display engine accepts the stock driver's display protocol; a separate broker presents owned frame copies and handles input. The owner requires guest flips to complete according to guest semantics while copies select the newest frame independently. The broker owns cursor composition, including inversion. Its absence or silence must not freeze guest display progress. [architecture] [display] [rulings]

Invariant: Guest flip completion is independent of a stalled viewer; frames and credits remain bounded; the broker never receives arbitrary guest memory as an owned frame.

Test: Display engine tests, plane-output oracles, socket simulations and pacing models exercise flip order, slow brokers and pixel formats. [display] [planes] [pacing]

What the test could miss: A fake broker cannot establish compositor, DMA-buffer or colour behaviour on every desktop. Correct pixels do not prove correct flip latency, and a correct cursor does not prove safe frame ownership. [pacing] [security]

5 What the evidence establishes

5.1 A result belongs to a cell and a revision

CODE READ. The status-detail document warns that much of its main body is September history. The README and newer per-topic records add Windows, display and relay results. This paper therefore does not copy a historical heading such as "Windows: research only" into a current support claim. Conversely, a later source audit does not rerun an earlier hardware test. The following table deliberately retains the recorded revisions. [status] [readme]

Cell	Revision	Recorded scope; repetition limit
Linux / TU116	`1915bd71`	Gates 9/9, bare and thin 30/30; one recorded merge-bar campaign. Ladder at `a295e2ce`, described as the same code. No timing spread claimed. [turing]
Linux / AD104 workstation	`fc2dadc4`	Gates 9/9, bare and thin 30/30; ladder 4/4 at `d40dafbf`. Distinct checkout revisions are preserved. [families]
Linux / GB205	`d40dafbf`	Gates 9/9, bare and thin 30/30, ladder 4/4. One recorded cell, not a Windows pass. [families]
Linux / GA104	`714ec29d3`	Host and guest 595.91.07; gates 9/9 plus gate 10, bare and thin 30/30, one box/cell. [requirements] [sweep]
Windows / AD104 desktop	`eb9f2aa3`	Stock 580.88 application matrix; later relay session at `89fc8a222`. Completion, recovery and output grades differ by app. No full-matrix pass at the paper baseline is asserted. [readme] [winmatrix]

MEASURED (reported). The family records above are dated 2026-09-30; the GA104 record is dated 2026-10-24. Pass counts enumerate tests or arms, not independent boots of the entire workload. A 30/30 thin suite boots separate arms; it is not a thirty-boot confidence interval for a multi-application desktop. The Linux family campaigns used nested rented hosts and open 580.159.04 drivers unless the cell says otherwise. [turing] [families] [sweep]

CODE READ. The public requirements list additional earlier Linux runs on GA106, GA102, AD106 and GB203, and the README mentions an AD102 Linux guest with driver 590. Windows desktops on AD106 and AD103 used a borrowed display page. The requirements explicitly say that no single current per-die table is documented. This is evidence fragmentation, not permission to call all neighbouring dies supported. Hopper and datacentre families remain source-derived or refused cases, not hardware qualifications. [requirements] [readme]

5.2 Correctness results and their sample units

MEASURED (reported). The recorded Linux application matrix reports 61/65 passing applications and six stream probes, with output comparisons to the same host; its source is an earlier GA106 campaign, not the paper baseline. The four remaining failures need managed-memory demand paging. The record of 100 sequential processes applies to one persistence-mode boot. These are useful workload and lifetime samples, but neither is a soak distribution. [status] [apps] [readme]

MEASURED (reported). The headless graphics record reports 38/38 items, including 31 byte-identical outputs, at `d06833f0` on a GA104 box. The video lane reports byte-identical encode/decode outputs on a GA106. These compare content, which is stronger than a renderer name; they do not cover every display mode, all video families or later revisions. [status] [video]

CODE READ. The driver matrix varies host and guest independently. Several guest versions passed a CUDA ladder against a fixed host, and several hosts passed the gates, thin suite and ladder against a fixed guest. The public guide preserves the versions and revisions and warns that the raw 30-arm suite is graded for 580.x guests. A decoder's support range and a tested application range are different sets. [requirements] [driverdesign]

5.3 A performance row with actual spread

MEASURED (reported). At device revision `e5ff45ff`, the AMD EPYC 7452 nested host with RTX 3060 and 580.159.04 ran eager Qwen2-0.5B decoding. For 512-token warm decode, the summary gives **12.62 tokens/s [12.40, 12.94],** $n = 3$ in the guest and **43.98 [43.59, 44.63],** $n = 3$ on the host, ratio **0.287**. These are the table's reported aggregates and ranges, not a new estimate or confidence interval. The associated text-equality checks matter separately from throughput. [llmmethod] [llmdata]

MEASURED (reported). The Windows performance summary, 2026-10-17, records three experimental Direct3D 11 Heaven runs at `134dcb54`: **299.1, 296.5 and 299.2 fps** with scan-off and lazy observation together, 1920×1080, Ultra, 260-second windows, stock 580.88 guest on the owner's AD104 host with 595.91.07. Its production-path samples at other listed revisions are slower. The record itself says these experiments were not a safe integrated lifecycle qualification and did not have an application matrix. Their combined gain cannot be assigned to just one optimisation. [winperf]

INFERRED. These examples support two narrower conclusions: large batches and small launches need separate measurements, and a fast renderer can coexist with a correctness or latency problem under mixed use. They do not establish a universal "virtualisation overhead" number. Nested versus non-nested execution, host load, guest workload and baseline revision all change the denominator. [linuxperf] [winperf]

5.4 The Worker GPU gate: useful, unattested and owner-controlled

CODE READ. GitHub requests a check through a Cloudflare Worker. A Durable Object ledger reserves liability, tracks owned instances, deadlines and spending, and coordinates cleanup. The provider account key stays in the Worker. Identity checks bind the workflow, owner triggering actor, signed run attempt and tested SHA. A scoped box capability can report only within its run contract. Cloudflare Access reduces noise; it is not the content-approval boundary. [broker] [threat]

CODE READ. At the baseline's contributor policy, ordinary PRs target `next`; the GPU check belongs on the exact final `next` to `master` batch. The policy states CI is required on next and CI plus `gpu-vast` on master. This is the recorded policy, not an audit of live GitHub ruleset settings. Older triage or handoff statements about required checks are historical. [rules]

Invariant: Only the owner-approved SHA rents within the configured policy, and cleanup acts on ledger-owned instances.

Test: Broker unit/runtime tests and recorded provider campaigns exercise identity, accounting, results, retries and teardown. A real runtime matters because Node fakes cannot reproduce all Worker fetch behaviour. [threat] [broker]

What the test could miss: The rented machine and its operator can lie. A valid callback authenticates the run capability, not honest GPU execution. A PASS is a regression tripwire, not release provenance, an attestation or a security audit. A missed cron or failed provider deletion can also outlive an internal deadline. [broker] [threat]

6 The uncomfortable part

The following is the research ledger, not a collection of release promises. It distinguishes fixed defects, competing explanations, experimental code and unrun acceptance tests. The open-PR inventory at the end preserves the publication snapshot; an open PR is not evidence that all of its changes are absent from the baseline.

6.1 The multi-application stall: start with the dependency, not its name

CODE READ. The October investigation asks why Windows applications can stall or recover while individual workloads run well. The later mapping experiments use the Enscape arm of the SPECviewperf workload. This paper discusses the failure and its timings; it publishes no SPEC scores, viewset results or proprietary workload images. The experimental mapping line is distinct from the baseline's earlier relay fixes. [admission] [rmcost] [latency]

A useful causal chain has separate stages: Windows queues work; the relay observes demand; it publishes host work; the GPU processes a covering marker; host software observes that marker; the interrupt path notifies the guest; the guest scheduler observes progress. A silence at the end does not locate the missing or late step. The proposed diagnostic contract keeps these stages separate and explicitly refuses to identify a private host publication with a Windows packet without evidence. [latency] [walknarrow]

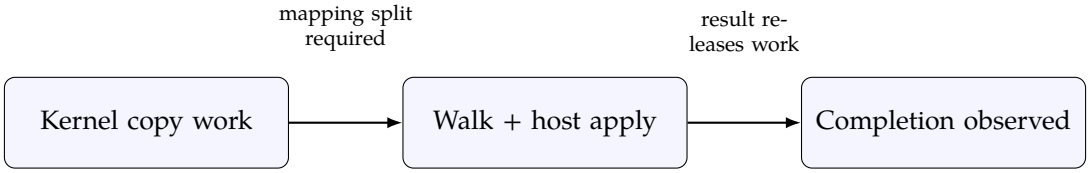


Figure 15: A serial dependency can resemble a missing interrupt; clocks and packet identity decide.

6.2 Timeline: defects that really existed, and explanations that did not survive

1. **CODE READ.** The October 9 invalidate incident left a guest trigger incomplete: run 223 counted 3,138 invalidates and 3,137 clears. The owner ruling records a guest polling that trigger indefinitely. Clearing over absent guest leaves became a policy; refused host unmap or host invalidate was explicitly not covered by that exception. A later reservation-edge defect was Kayfabe's mapping failure, not invalid guest tables. These are documented predecessors, not proof of the later Enscape cause. [rulings]

2. **CODE READ.** The earlier USER-ring completion gate used GET as a witness. The event-loop record says hardware could fetch beyond a dependency before the dependent work ran. A tagged processed-reference marker replaced that proof. The attractive explanation "GET passed it, so it executed" was too strong. [eventloop]

3. **MEASURED (reported).** The README records an earlier relay-handoff campaign with 0 stalled pairs out of 26 after the fix, against 3 out of 12 before, plus a later Windows mixed-app session of more than fifteen minutes. These are the README's scoped reports at `89fc8a222`; the paper does not claim a population-wide stall rate or transfer that result to the later sparse experiment. [readme]

4. **CODE READ.** A later Translated-slot contention defect consumed a wake while another thread held the channel slot. The token could return idle with no owner or future pump. The merged recovery retains the owed ring; behavioural tests demonstrated cases that source-shape guards

had missed. Its design record also says a Turing run still had Windows reset storms after the narrow wake defect was fixed. [owed]

5. **CODE READ.** The first sparse-admission implementation cloned and restored the whole reservation tree on failure, returned an uncounted hard error for occupied coarse geometry, and could resend a failed acknowledgement on later walks. Local undo, explicit decline accounting and retry metadata addressed those software mechanisms. Their CPU tests were not an Enscape acceptance run. [admission]

6. **MEASURED (reported).** At sparse-admission revision `6d95870c7`, runs 7330 and 7350 showed reservation churn near the 8,192-block policy boundary. In run 7330, over approximately 66 seconds, made objects rose by 241,584 and freed objects by 241,583. The record also reports a 1.94-second apply with roughly 59,000 reservation verbs. This refutes the simple explanation that every long apply must be one slow native call. [rmcost]

7. **MEASURED (reported).** That same record puts run 7330's first event 153 near the Enscape start while recent VA heartbeats were idle. Missing sparse coverage and declines coincided with onset. This argues against treating earlier reservation-churn latency as a complete account of that particular first reset. It does not prove missing coverage caused it. **INFERRED.** The hypotheses require separate interventions. [rmcost]

8. **CODE READ.** Later scope tests passed on the original gate while harvesting a published marker and writing GET back before the PUT admission gate. They falsify the narrow claim that this gate alone prevents completion observation. They do not falsify delayed worker scheduling, lock contention or a dependency before publication. [walknarrow]

The timeline is deliberately non-monotone: a real lost wake, a throughput defect and a missing mapping can all occur during one campaign. Finding the next defect does not retract a previously demonstrated one, and fixing the previous one does not close the new case.

6.3 What the later walks actually say

MEASURED (reported). The public split-latency note attributes **297,495 of 318,747** walks in boot 7321 and **461,389 of 490,686** in boot 7391 to one privileged Translated copy-channel token. These are roughly **93–94%**, with one count pair per boot, not a sampled estimate with a confidence interval. The note is a secondary report supplied to the implementation review; this paper has not audited the raw host traces. [latency]

MEASURED (reported). The scope-narrowing note reports a **2.43-second apply** and a **2.529-second completion gap** in boot 7391 at `64c50ad4`. With raised diagnostic TDR thresholds, that Enscape arm reached its time limit without event 153. The note reports completion intervals following walk ends in boot 7321. These observations support a *late rather than permanently lost* account for that arm; they are not a timeout-based fix, a broad timing distribution, or a one-to-one packet-identity proof. [walknarrow]

INFERRED. A mostly serial channel that asks for a mapping split, waits for its result and then issues the next request can produce one observed completion near each walk end. That is consistent with the timing report. Identifying the Windows scheduling node with that exact host channel still requires ticket-level correlation. Correlation of two periodic series is not the missing identity. [latency] [walknarrow]

CODE READ. A split requests a fresh post-admission walk because releasing later work against an earlier snapshot could expose stale translations. Same-target tickets admitted together can share a walk; a request arriving during an active walk belongs to a later submission. A channel that waits for each result before issuing the next request supplies nothing concurrent to coalesce. [latency]

Acceptance still absent from the cited record: a full production-profile, default-timeout, sparse-

twin-enabled Windows boot that passes the failing mixed workload. The software validation notes explicitly retain pending host acceptance and the default-OFF sparse policy. A longer diagnostic timeout cannot fill that gap. [sparse] [budget] [latency]

6.4 How our explanation could be wrong

First fork: publication delay versus observation delay. A packet can spend its time waiting to be published, executing on hardware, or complete while the observing worker is busy. The native marker has no GPU-write timestamp in the current diagnostic contract; `marker_gpu_ns=None` is intentional. Publication-to-observed-marker therefore combines execution and observation latency. Calling all of it "GPU time" would be false. [latency]

Second fork: CPU scope versus shared hardware. A VM-wide admission gate can delay an unrelated ring before publication. Narrowing it by trusted host mutation target removes that particular dependency. It cannot remove shared engine scheduling, a global host RM lock, or necessary same-target ordering. If the same stalls persist with unrelated holds removed, that does not show the scope fix was inert; it shows the intervention did not isolate the remaining bottleneck. [walknarrow]

Third fork: trigger, victim and common cause. A kernel copy channel can request most walks while an application merely suffers from them. Alternatively, the application's allocation pattern can cause both sparse work and scheduler distress. Per-channel walk counts alone cannot distinguish those stories. The decisive observation needs request, walk start, apply end, readiness, consumption, publication and notification in one life and clock domain. [latency]

Proposed falsifier, not a run result: repeat the same workload and fixed image with native publication/marker/vector histories and guest event tracing. Include a diagnostic `TdrDelay=60` arm only as an observation window, alongside unchanged default thresholds. If long completion gaps occur without a corresponding pre-publication split hold, reopen relay/observation and GPU scheduling explanations. If changing sparse coverage alone changes onset while latency remains unchanged, prioritise the semantic mapping hypothesis. Record missing coverage and incomplete timestamps rather than dropping those samples.

6.5 Sparse backing: semantic fidelity has a resource cost

CODE READ. A sparse guest page is not the same as an invalid unmapped address. The selective sparse-twin experiment exists to reproduce that distinction using host reservation objects. It stays default OFF. Here sparse means an intentionally unbacked mapping whose reads return zero and whose writes are absorbed; plain invalid is a different state that faults. A native micro-oracle can show that the sparse state changes a fault outcome without proving a long-lived Windows desktop remains correct. [sparse] [sparsepr]

CODE READ. The newer policy scales finite per-space and per-session object budgets from the configured guest framebuffer and counts missing coverage separately from cumulative declines. Its interval tracker uses an explicit conservative upper bound on overflow; it must not turn unknown missing coverage into a zero gauge. A permanently excluded host-owned window remains excluded rather than repeatedly retried. These choices prefer faulting to inventing zero-filled backing or taking ownership of a host placement. [budget]

INFERRED. This is a design tradeoff that deserves an unfriendly name: the fallback can be closed for host authority while still incompatible with a guest that needs sparse semantics. A finite cap protects the host; it does not make the guest's request disappear. The acceptance question is whether supported workloads stay within the defined mode, not whether a counter has an appealing value. [budget] [rulings]

6.6 Soft budgets are not deadlines

CODE READ. The sparse layout allowance limits the work that may *start* in one slice. Already started replacements must finish or repair their original ownership. An individual native RM call is not software-preemptible. Ordinary map/unmap, invalidation, retirement and full teardown are not all covered by the optional-layout allowance. [rmcost] [budget]

MEASURED (reported). A later CPU simulation with 120-ms releases required 240 ms for release and mandatory repair, even after two budget bypasses were fixed. That result explicitly disproves a universal 110-ms expectation. The original assertion of at most one replacement overshoot was false for error cleanup that started additional work without checking the slice. These are simulated timings at the scope-repair checkpoint, not a hardware guarantee. [walknarrow]

CODE READ. The latest experimental continuation defers optional geometry until mandatory split results and wakes are released, rotates among target identities and stops between native verbs when a new split arrives. Mandatory repair still completes. Under continuous mandatory work, optional coverage may remain unhonoured; it is counted rather than hidden. No hard total-apply deadline or Windows cure is established. [latency]

6.7 Mirrors: retiring memory is an ordering protocol

CODE READ. A mirrored address space contains host placements, not merely a dictionary of guest addresses. The retirement PR delays freeing old backing until invalidation acknowledges that hardware can no longer use the old translation. A failed invalidate retains ownership. Its original proposal also keeps explicit unresolved scope tests rather than claiming a per-range gate exists. [retirementpr] [walkscope]

INFERRED. A mirror can be correct after every completed refresh and still produce excessive latency during a refresh. Conversely, narrowing an admission gate can be fast and wrong if it releases a source whose mapping is still changing. The experimental target-scoping argument is about the host object actually mutated, not an assumption that distinct guest roots never share table pages. Unknown/global requests retain conservative holds. [walknarrow]

The remaining fork is per-range certification: can unchanged source spans continue through an own-space refresh while changed spans permit only old, new or fault outcomes? The latest note marks that as design work. It needs a bounded change history, root-generation handling, overflow fallback and publication revalidation; a per-space flag alone is not that protocol. [latency] [walknarrow]

6.8 Pointer mode and lazy observation: the proof costs more than the optimisation

CODE READ. Pointer mode exists because repeatedly reading command bytes costs CPU time even when hardware can safely read the guest's own admitted commands. Registration and migration still require scanning boundaries. A window anchor is a fact about what was observed before a reply; retrospectively choosing the current PUT would weaken the proof. The lifecycle record corrects earlier boundary and mode-scope claims in place. [lifecycle]

CODE READ. Lazy observation additionally makes the guest-visible GET lag. The code compensates with fresh observations on waits, low credits, split segments, full-ring pressure and mode changes. Its argument is that staleness can only reduce credit or delay a gate, never create progress. The small-ring bound matters because a large benchmark ring can hide guest backpressure. [lifecycle] [lazycode]

INFERRED. These mechanisms are justified only if the whole lifecycle remains easier to test than an always-scanning path is to run. A local counter win is insufficient. One should demand small-

ring tests, hostile mutation schedules, migration at packet boundaries and native completion timing. The combined scratch performance arms do not reveal which half of lazy observation deserves its permanent complexity. [winperf] [lifecycle]

6.9 Relay debt and recorded disagreement

CODE READ. The public ruling records an evolution in the Windows mediation decision: an initial immutable-copy rule was superseded by the owner's clarification that native USER privilege plus restricted mappings can permit ordinary GP references unchanged. The earlier design required immutable command copies; the owner later permitted ordinary pointers under the native authority boundary. This is a recorded correction to the design, not evidence that host-owned writable memory became acceptable. Linux Passthrough's stricter mapping prohibition remains. [rulings] [migrationdecision]

CODE READ. The latest split note explicitly preserves the existing event-driven relay and introduces no 64-attempt budget. The new mapping work targets a different dependency. A convenient retry count would not prove that a lock holder hands progress back correctly. The contention test's finite fair suffix is an assumption about scheduling, not a product polling interval. [latency] [owed]

CODE READ. A separate retry-set relay proposal at `c452ca7cb` makes every loop iteration inspect outstanding state and keeps unsourced obligations spinning rather than waiting for a holder notification. It also experiments with returning ring-slot credit at software read progress instead of hardware fetch progress. Its design note records changed or unresolved model expectations and no hardware run at that checkpoint. Simpler loop structure is an argument to test, not evidence that either credit semantics or Windows behaviour is better. [alternativerelay]

UNVERIFIED. This paper found no public repository record sufficient to attribute a current coordinator preference between every historical relay alternative. It therefore invents no personal dissent. The practical competing proposals that are recorded are native relocation, restricted mediation and an optional host-cooperation tier; their acceptance obligations are more useful than an attributed opinion without a source. [migrationdecision]

6.10 Poor shapes we should be willing to replace

Global publication dependencies. INFERRED. Holding every relayed ring while one target changes gives one workload an avoidable latency path into others. The scope experiment addresses only this part. Owner policy still requires unchanged VAs to remain usable and prohibits solving ordering with a GPU pause. [walknarrow] [rulings]

Work hidden behind a soft bound. CODE READ. A "100 ms slice" that excludes repair, mapping and teardown is easy to overstate. The newer notes name these exclusions. Owner policy requires prompt invalidate completion; changing a diagnostic timer is not compliance. [rmcost] [rulings]

Control-plane scaling. CODE READ. The open indexing PR describes a partial correction to quadratic RM free paths. It does not claim all graph or hostile-flood costs are linear. A cap is a finite memory bound, not a useful throughput bound. [indexpr]

Classification by convenient identity. CODE READ. The PR catalogue includes separate privilege-stamp and mirror-classification work. Guest-handle bit patterns are not an authority proof; the rule is native privilege and validated ownership. Integration must preserve that rule across old branches, not merely fix the newest path. [rulings]

Flag accumulation. CODE READ. The flag inventory and cooperative-tier plan distinguish measurement switches from product modes. A correctness feature that requires an undocumented

diagnostic flag has not met the stock-path requirement. The sparse switch currently labels an experiment, not a supported mode that must be enabled. [flags] [cooperation] [sparse]

Evidence scattered across heads. CODE READ. Status pages, exact-SHA traces and open stack PRs often refer to different revisions. The original status page warns about its own age. A correction in one page cannot silently update a binary or a downstream summary. [status] [handoff] [old]

6.11 Per-die constants: derive first, then admit the coverage gap

CODE READ. The public provenance policy permits computed values, pinned open sources, unprivileged host queries and source-tagged hardware measurements. Closed-binary material is not a public data source, build dependency or source of interface layouts. The policy itself is draft wording awaiting review, and it explicitly does not claim a clean-room process. This paper carries no findings, values or citations derived from closed binaries. [provenance] [credits]

CODE READ. The constants strategy says to derive or compute first, then ask the unprivileged host, then measure on the exact die and driver when necessary. A family compression needs both agreement across measured dies and a mechanism. Agreement of adjacent dies is only "consistent so far". An unmeasured die or branch gets a defined, tested refusal or fallback, not a neighbour's unlabelled row. [constants] [rulings]

CODE READ. Coverage is not all on the baseline. The strategy identifies published constants and display-capability consumers on separate experimental heads while the baseline retains older paths. Some Windows dies use borrowed display data; Blackwell Windows remains a set of specific failures and proposed field derivations. Hopper and datacentre cases have no hardware cell in the guide. A table loader existing is not proof that every caller uses it or that the fallback runs real applications. [constants] [requirements]

INFERRED. A demanding external test is to remove every borrowed optional bit for an unmeasured die and run the exact acceptance workload. If the fallback only boots but cannot complete ordinary applications, "clean mode" was a name rather than a product configuration. This follows the owner's explicit fallback requirement, not a new promise of support. [constants]

6.12 Security: unprivileged is a capability limit, not containment evidence

CODE READ. Kayfabe trusts the host NVIDIA driver, GPU and VMM adapter. It has no independent security audit or penetration test. The process has no Kayfabe-owned sandbox; deployment must confine it. GPU USER privilege and per-channel VA spaces are central because the guest does not own the physical device. The host IOMMU is a separate host protection layer and does not provide per-VM identities on this shared physical GPU. [security] [faq]

MEASURED (reported). The physical-operand oracle at `8c084ab7`, RTX 4070 and 595.91.07, reports that USER copy channels refuse physical operands while virtual operands are delivered. That is strong evidence for that tested boundary. It is not an audit of every engine, privilege transition, host-driver bug or same-UID cross-client path. [physevidence] [faq]

CODE READ. Deployment recommendations include one UID per VM, dropping capabilities and confining the whole VMM. The public security page records resource gaps from a dated source review: GPU time, NVENC sessions, host BAR1, RM call rate and logging were not all bounded per VM. It explicitly says that review has not been repeated and later code may address parts. This paper does not turn that historical list into a fresh vulnerability census. [security]

6.13 Fault containment, notification noise and fairness

CODE READ. GPU-wide non-stall notifications can wake every registered VM. Extra interrupts and timing information about engine use are accepted tradeoffs; dropping a real wake would break correctness. A guest checks its own completion value after a spurious notification. Optional pacing can delay a signal but must not erase the debt. [faq] [rulings]

UNVERIFIED. The effect of a guest-triggered fault on other VMs sharing the same GPU remains unmeasured in the public guide. Isolation of normal addresses does not establish reset or fault containment. Likewise a finite framebuffer allocation does not establish fair scheduling, encoder access or host-call service under an adversarial peer. A useful claim needs a live victim with retained work across the fault, not just a new process started afterwards. [security] [faq]

6.14 Performance honesty: denominator, placement and missing samples

CODE READ. The public Linux performance baseline distinguishes nested rented hosts from the owner's physical host. It records why the obvious pre-Windows comparison revision could not run with the current host driver, so the chosen baseline already contains earlier Windows work. A before/after delta using it cannot measure all Windows cost. [linuxperf]

INFERRED. A throughput ratio is useful only for the named workload and placement. Comparing a GPU reading host system memory with one reading VRAM answers a placement question as well as a virtualisation question. An application that batches a token's work cannot grade eager launch overhead. Three repeated runs in one boot do not estimate boot-to-boot variability. The older paper's correction record contains exactly these denominator and control failures. [old] [linuxperf]

CODE READ. The display summary warns that average Heaven FPS omits low-frame behaviour and that its native reference was not repeated with every experimental arm. A fast scratch build is not a passing integrated lifecycle. No proprietary 3DMark, Steel Nomad or Geekbench score is reproduced here. No SPEC numerical score is needed to describe the stall. [winperf]

6.15 Unsupported, partially measured and explicitly postponed

Multiple GPUs. CODE READ. A Linux thin guest used two distinct GPUs, sequentially and concurrently. CUDA in that guest, UVM across devices, peer access, Windows and choosing a display GPU remain outside that result. Calling multi-GPU simply "unbuilt" would erase real evidence; calling it a complete product would overstate it. [status] [rulings]

Other VMMs and arm64. CODE READ. The owner records these as goals after family coverage. QEMU is the implemented adapter. A cross-build or historical pure-core ARM test does not establish today's memory/interrupt/reset shell on arm64. [rulings] [requirements]

Blackwell Windows. CODE READ. The guide records GB206 Code 43; open proposals cover display fields, constructor admission, native probes and readiness. A successful Linux GB205 cell is not a Windows GB206 or GB202 pass. [requirements]

Driver unload and recovery. CODE READ. The reset and gap documents distinguish device life, in-guest reset and reboot. A correct cold boot cannot establish repeated driver unload/reload, kill-during-work recovery or generation reuse. Some reset work exists; its presence is not a complete lifecycle acceptance. [reset] [gaps]

Suspend. MEASURED (reported). Linux S3 passed three cycles at `559db30c`; the status record reports Windows wake failures. Thus S3 is neither globally absent nor globally working. The FSP-family resume path remains a named gap. [sleep] [handoff]

Display across reboots. CODE READ. Reboot scrubbing and boot-layer taint are distinct policies: a reboot must scrub before the next OS life, while a driver reset must not pretend to create a new trust boundary. The owner's host-allocation isolation premise is explicitly accepted, not independently verified by the project. [rulings] [reset]

Other long tails. CODE READ. Multi-head, advanced display fidelity, vIOMMU addressing, demand faults, peer resources, container packaging and a rootless launcher need their own evidence or implementation. The gap catalogue preserves the issue distinctions; an untested configuration is not a supported fallback. [gaps] [faq]

6.16 Licensing and provenance are unfinished release work

CODE READ. The repository states Apache-2.0 OR GPL-2.0-or-later with exceptions. Some nvproxy-derived files remain Apache-only; the credits say a QEMU binary linked from the current tree is not yet distributable under the GPL. The broker has its own notice. Vendor driver, firmware and OS binaries are not supplied by Kayfabe. These are the project's stated notices, not this paper's legal interpretation. [credits] [license] [notices]

CODE READ. Tests obtain vendor software under its own terms. The public downloads note is a draft and records unresolved installer/automation questions. Building a sealed Windows cache or automating an installer is engineering work separate from permission to redistribute it. The policy calls for review of provenance wording and the full history before release. Nothing in a successful benchmark or build settles those questions. [credits] [provenance]

6.17 Process risk: one maintainer, many agents, fallible instruments

CODE READ. The repository's workflow assigns an owner approval boundary and uses agents for research, implementation and review. It requires falsifiers, expected-red tests, multiple mutation killers, exact revisions and attribution. These mechanisms expose claims to tests; they do not establish reviewer independence when the same mistaken model informs both code and oracle. The predecessor records an independent-looking reference tracker with the same logical omission as the implementation. [rules] [old]

INFERRED. The maintainer remains an integration bottleneck: choosing among many experimental heads requires understanding both the code and which hardware receipts still apply. A machine-written summary can amplify a confident commit message that was never evidence. The remedy is source and experiment review, not counting how many reviewers repeat it. The old paper's corrected object-graph hypothesis illustrates that amplification. [old]

CODE READ. The current PR policy batches real-GPU validation on master and keeps next for integration. That controls cost, but makes exact final-tree qualification essential. A passing parent plus a passing child on another date is not a passing merge. Open expected-red PRs and ignored contracts are useful debt records only while their status remains visible. [rules] [owed]

6.18 Research inventory at the publication cutoff

CODE READ. The following inventory covers the 86 open PRs captured when writing began, before this paper's own PR. It includes ordinary and draft PRs, older experiments and a pending batch. Each entry states its goal, snapshot state and a *proposed* falsifier or acceptance result. A falsifier here is the paper's research question, not a claim that the PR author preregistered or ran it. The public PR number and immutable head identify the source. Names from PR titles are translated into plain descriptions; historical labels are decoded in the appendix.

State "open" means GitHub state at capture. "Head already in baseline" is a Git ancestry check, not a claim that a later replacement is identical. Body-reported validation, when mentioned, has not been rerun. Conflicts, policy decisions and full-profile hardware qualification remain integration work. The separate experiment notes cited above may contain later work beyond the older sparse/mirror PR heads.

PR 5. Archive unused predecessor crates without breaking the grader **CODE READ.** Snapshot: draft, open. Public proposal; head c9f00664b439. **Proposed falsifier:** The raw-client dependency closure no longer builds or a product crate acquires a legacy dependency.

PR 7. Model shared Turing display semaphores **CODE READ.** Snapshot: draft, open. Public proposal; head 320b3317033b. **Proposed falsifier:** Native acquire/release ordering differs or one window consumes another window's event.

PR 8. Compose address-named Blackwell windows inside the store **CODE READ.** Snapshot: draft, open. Public proposal; head 598ca60f8b76. **Proposed falsifier:** A window escapes the store or a native pixel/ordering reference disagrees.

PR 9. Wire exact AD102 Windows initialisation facts **CODE READ.** Snapshot: draft, open. Public proposal; head c3f9c13d5576. **Proposed falsifier:** Native same-die replies or Windows acceptance disagree with the supplied fields.

PR 10. Admit measured Turing channel and driver layouts **CODE READ.** Snapshot: draft, open. Public proposal; head 728e6febde5e. **Proposed falsifier:** A same-die driver cell disagrees or an unrelated branch inherits the layout.

PR 12. Wire measured GB206 display capabilities across both guests **CODE READ.** Snapshot: draft, open. Public proposal; head 24b6a4256ebd. **Proposed falsifier:** Exact-head Linux or Windows still fails its application and display acceptance.

PR 13. Recover guest command results after agent disconnect **CODE READ.** Snapshot: draft, open. Public proposal; head 8aa16cd64557. **Proposed falsifier:** Recovery reexecutes a side-effecting command or associates output with another request.

PR 14. Follow early queue movement in the observer **CODE READ.** Snapshot: draft, open. Public proposal; head 631f087e34e2. **Proposed falsifier:** The observer misses a real queue relocation or reads an unvalidated candidate.

PR 15. Compare derived display capabilities to native oracles **CODE READ.** Snapshot: draft, open. Public proposal; head 886f885c5ffa. **Proposed falsifier:** A mutated wrong capability still passes the claimed behaviour-level oracle.

PR 16. Export bounded ordered firmware metadata **CODE READ.** Snapshot: draft, open. Public proposal; head 9251e45c4541. **Proposed falsifier:** An overflow looks like complete coverage or ordering changes during export.

PR 17. Share CPU views of the backing store **CODE READ.** Snapshot: draft, open. Public proposal; head 75b247c07e65. **Proposed falsifier:** View sharing breaks the Linux fast gate or permits stale topology/lifetime access.

PR 18. Follow validated public firmware mailbox indirection **CODE READ.** Snapshot: draft, open. Public proposal; head 20e0ddc4b063. **Proposed falsifier:** An unvalidated pointer becomes a source or a legitimate relocation is missed.

PR 19. Require a fresh event for GPU-walk completion **CODE READ.** Snapshot: draft, open. Public proposal; head 2bf43d609ff5. **Proposed falsifier:** An old completion can acknowledge a newly submitted walk or a fresh result is lost.

PR 20. Stop fast guests without forced process kills **CODE READ.** Snapshot: draft, open. Public proposal; head 21b72582ccdb. **Proposed falsifier:** The normal stop leaves an owned process alive or kills a process outside the run.

PR 21. Retain full startup and teardown trace alignment **CODE READ.** Snapshot: draft, open. Public proposal; head 3008bc8346bd. **Proposed falsifier:** A truncated trace looks complete or the aligner pairs different device lives.

PR 22. Trace replies actually posted to the guest **CODE READ.** Snapshot: draft, open. Public proposal; head 4d83e661fcc6. **Proposed falsifier:** A queued but unposted reply is counted as delivered or a posted reply is omitted.

PR 23. Align firmware heartbeat time to the guest timer **CODE READ.** Snapshot: draft, open. Public proposal; head ac01b7bd306d. **Proposed falsifier:** A changed offset breaks monotonicity or produces a heartbeat in another clock domain.

PR 24. Add measured GB205 driver-specific capability words **CODE READ.** Snapshot: draft, open. Public proposal; head 5b8dd3a5ff43. **Proposed falsifier:** Repeat measurement or another branch differs, yet the consumer silently reuses the row.

PR 25. Bound metadata traces across guest lifetimes **CODE READ.** Snapshot: draft, open. Public proposal; head fee2ac9336ee. **Proposed falsifier:** Reset evades the cap or old-life records are attributed to a new guest.

PR 26. Select broker guest image and ABI explicitly **CODE READ.** Snapshot: draft, open. Public proposal; head c07f8e8ad470. **Proposed falsifier:** An implicit image or driver choice overrides the requested test cell.

PR 27. Stop only the owned interactive QEMU safely **CODE READ.** Snapshot: draft, open. Public proposal; head d9aec37d5afe. **Proposed falsifier:** Shutdown targets a foreign PID or returns while the owned guest is still running.

PR 28. Synchronise bounded guest-agent file staging **CODE READ.** Snapshot: draft, open. Public proposal; head 5a300e75d3b1. **Proposed falsifier:** A lost acknowledgement replays a write or a partial transfer is used as a complete file.

PR 29. Derive virtual Blackwell tiles and display assignments **CODE READ.** Snapshot: draft, open. Public proposal; head 712a0a4755c4. **Proposed falsifier:** Native acceptance and virtual assignment bounds disagree or an unimplemented feature is enabled.

PR 30. Isolate sensitivity to one Blackwell log-buffer refusal **CODE READ.** Snapshot: draft, open. Public proposal; head 73af410b683c. **Proposed falsifier:** Matched refusal and served arms leave the outcome unchanged while reachability is proven.

PR 31. Derive exact public display layouts for a driver tag **CODE READ.** Snapshot: draft, open. Public proposal; head 98b49a1a4fb6. **Proposed falsifier:** The native layout differs or the table is selected for a different driver version.

PR 38. Preregister object-binding experiments **CODE READ.** Snapshot: open, non-draft. Public proposal; head ed81e56f37f0. **Proposed falsifier:** An arm is interpreted without its positive control or an outcome contradicts its recorded prediction.

PR 46. Scope a Windows allocation diagnostic to its measured die **CODE READ.** Snapshot: draft, open. Public proposal; head 90364d9d86a6. **Proposed falsifier:** An unsupported die or driver inherits the diagnostic answer as a product capability.

PR 50. Handle a legacy cursor format and bounded diagnostics **CODE READ.** Snapshot: open, non-draft. Public proposal; head 61dc81b73aa2. **Proposed falsifier:** Cursor pixels or inversion differ from the native reference, or composition violates broker ownership.

PR 51. Derive display DMA bounds from the class **CODE READ.** Snapshot: draft, open. Public proposal; head 70afcac481a8. **Proposed falsifier:** A boundary operand reaches outside the owned store or a valid last element is refused.

PR 52. Observe public Blackwell firmware bootstrap indirection **CODE READ.** Snapshot: draft, open. Public proposal; head b9e2462811ef. **Proposed falsifier:** A moved queue is missed or an unvalidated location becomes a capture source.

PR 53. Admit generated Blackwell video-decode constructors **CODE READ.** Snapshot: draft, open. Public proposal; head aa8fdea9cb0e. **Proposed falsifier:** Decode content or errors differ from the native reference despite successful construction.

PR 55. Admit the public Blackwell optical-flow constructor **CODE READ.** Snapshot: draft, open. Public proposal; head 844eb167f3a4. **Proposed falsifier:** The native constructor or an actual flow workload fails despite an accepted class.

PR 56. Review silent registration of a power-management notifier **CODE READ.** Snapshot: draft, open. Public proposal; head c58989dab89e. **Proposed falsifier:** The guest later requires a hardware action that the accepted inert registration did not perform.

PR 57. Add one independently measured GB202 capability scalar **CODE READ.** Snapshot: draft, open. Public proposal; head 1bba760c4c6f. **Proposed falsifier:** A same-die repeat disagrees or the row is applied to an unmeasured driver branch.

PR 58. Admit public Blackwell kernel-graphics report releases **CODE READ.** Snapshot: draft, open. Public proposal; head d04f8ae9ad9a. **Proposed falsifier:** The report value or completion order disagrees with the native oracle.

PR 59. Derive Blackwell post-composition tile capabilities **CODE READ.** Snapshot: draft, open. Public proposal; head 680efbb32c77. **Proposed falsifier:** A native supported tile is refused or the virtual device advertises a tile it cannot execute.

PR 60. Derive Blackwell output-table capacity **CODE READ.** Snapshot: draft, open. Public proposal; head f62ec368f68a. **Proposed falsifier:** The advertised table size exceeds the emulated decoder or rejects a valid native boundary.

PR 61. Use allocated family classes in native graphics fixtures **CODE READ.** Snapshot: draft, open. Public proposal; head 97dc65cc8ec9. **Proposed falsifier:** The fixture fails because it uses a foreign class, or false success comes from no engine work.

PR 62. Keep process ownership during relay qualification **CODE READ.** Snapshot: draft, open. Public proposal; head fd23ab1c5933. **Proposed falsifier:** A harness stop kills a foreign process or loses the GPU lease while its owned process is live.

PR 65. Derive post-composition unit widths **CODE READ.** Snapshot: draft, open. Public proposal; head 296636c66264. **Proposed falsifier:** A boundary-value native transform disagrees or an impossible width is advertised.

PR 68. Prototype scoped GPU CI callbacks and endpoint routing **CODE READ.** Snapshot: draft, open. Public proposal; head ffd1d1d518e8. **Proposed falsifier:** Wrong-run or expired data is admitted, or a URL locator is mistaken for content approval.

PR 69. Derive colour-space precision from the open display interface **CODE READ.** Snapshot: draft, open. Public proposal; head 6e7cb6ff1bef. **Proposed falsifier:** A native boundary-value transform disagrees with the derived precision or field width.

PR 70. Record unresolved command-descriptor continuation choices **CODE READ.** Snapshot: draft, open. Public proposal; head 03715b7d3255. **Proposed falsifier:** A packet spanning descriptors is accepted without a consistent continuation and lifetime rule.

PR 71. Propagate real translated-channel errors to the guest **CODE READ.** Snapshot: draft, open. Public proposal; head 1c6452a276d7. **Proposed falsifier:** A host error becomes success, loses its notifier, or contaminates another live channel.

PR 72. Document normal colour decoding and optional diagnostics **CODE READ.** Snapshot: draft, open. Public proposal; head 295419c84a54. **Proposed falsifier:** Following the documented default requires a diagnostic flag or produces a different transform.

PR 73. Require native privilege stamps for kernel-channel authority **CODE READ.** Snapshot: draft, open. Public proposal; head 69ef5973d044. **Proposed falsifier:** Guest-controlled identity bits grant privileged execution without a trusted allocation stamp.

PR 74. Split all-space walks at the ABI batch boundary **CODE READ.** Snapshot: draft, open. Public proposal; head e6a0d2b8185d. **Proposed falsifier:** The first space beyond the batch limit is omitted, duplicated or prematurely acknowledged.

PR 75. Expose root-move invalidation and withdrawal questions **CODE READ.** Snapshot: draft, open. Public proposal; head 01de2fed491a. **Proposed falsifier:** A moved or removed root can acknowledge success while the wrong host translation remains live.

PR 76. Admit pointer descriptors using their hardware format **CODE READ.** Snapshot: draft, open. Public proposal; head 468a35c9d845. **Proposed falsifier:** Valid hardware descriptor bits are refused or reserved bits create extra address authority.

PR 77. Preserve scan fairness and trailing rescan wakes **CODE READ.** Snapshot: draft, open. Public proposal; head 1c510cb68e94. **Proposed falsifier:** One hot token starves another or the final publication is stranded without a future wake.

PR 78. Prevent stale ring-tail arithmetic from poisoning publication **CODE READ.** Snapshot: draft, open. Public proposal; head 750820e95d88. **Proposed falsifier:** A wrapped or stale tail underflows into a false capacity or corrupts a live producer.

PR 79. Bound streamed command classification on workers **CODE READ.** Snapshot: draft, open. Public proposal; head e018e784cecb. **Proposed falsifier:** A hostile packet stream consumes unbounded work or a bounded continuation loses parser state.

PR 80. Index expensive resource-manager free paths **CODE READ.** Snapshot: draft, open. Public proposal; head 5532cdc88ba8. **Proposed falsifier:** A growing object graph still produces quadratic cost in the targeted free path.

PR 81. Cap live CPU mapping windows before native placement **CODE READ.** Snapshot: draft, open. Public proposal; head fdccc15e8692. **Proposed falsifier:** Concurrent admissions exceed the cap or failure rollback leaks ownership or a slot.

PR 82. Expose invalidate snapshot drift with failing contracts **CODE READ.** Snapshot: draft, open. Public proposal; head 2363313c8c28. **Proposed falsifier:** A root change can settle against a stale snapshot; ignored contracts are not a runtime fix.

PR 83. Bound guest-driven diagnostics at one admission point **CODE READ.** Snapshot: draft, open. Public proposal; head fa012c5b4d96. **Proposed falsifier:** A flood bypasses the cap through another path or a refusal hides unrelated progress.

PR 84. Prevent unsupported device unplug on the current topology **CODE READ.** Snapshot: draft, open. Public proposal; head 2165486e0a21. **Proposed falsifier:** The guest can still unplug it or legitimate reset/shutdown loses the required lifecycle event.

PR 85. Query display capabilities through unprivileged host requests **CODE READ.** Snapshot: draft, open. Public proposal; head 124a37ce3dba. **Proposed falsifier:** The query needs privilege or a whole physical answer enables an unsupported virtual field.

PR 86. Repair combined-candidate review findings **CODE READ.** Snapshot: draft, open. Public proposal; head ee80c5eb9589. **Proposed falsifier:** A hostile-input or ownership regression remains reachable in the exact integrated candidate.

PR 87. Check design status and identify dead code **CODE READ.** Snapshot: draft, open. Public proposal; head e50cf4da55d1. **Proposed falsifier:** A live proposal is deleted as dead or a source-status correction claims implementation exists.

PR 88. Bound trap-side doorbell batches **CODE READ.** Snapshot: draft, open. Public proposal; head c63f66cdbc86. **Proposed falsifier:** An admitted large delta blocks a vCPU or the worker fallback loses or duplicates entries.

PR 90. Seed the copy decoder from the allocated host class **CODE READ.** Snapshot: draft, open. Public proposal; head 9490d7227e26. **Proposed falsifier:** A family-specific object is decoded with the wrong method set or privilege checks weaken.

PR 91. Test translated errors against a live Linux victim **CODE READ.** Snapshot: draft, open. Public proposal; head 67df34aff3c7. **Proposed falsifier:** An induced attacker fault silently succeeds, corrupts the victim, or poisons later guest work.

PR 92. Handle mirrored output lookup tables in the colour path **CODE READ.** Snapshot: draft, open. Public proposal; head e636236cf346. **Proposed falsifier:** Native colour controls and captured output disagree or the target UI operation still freezes.

PR 93. Derive virtual display fields before consulting host facts **CODE READ.** Snapshot: draft, open. Public proposal; head e4cec12b1b31. **Proposed falsifier:** A physical host capability enables a virtual feature that the emulated display cannot perform.

PR 95. Observe actual Windows adapter readiness independently **CODE READ.** Snapshot: draft, open. Public proposal; head 1ac2b51d56ca. **Proposed falsifier:** An enable-command return is confused with adapter Code 0 or the observer reads a stale instance.

PR 97. Register native completion listeners before submission **CODE READ.** Snapshot: draft, open. Public proposal; head f4879bda2499. **Proposed falsifier:** A completion arriving immediately after submission is missed or attributed to another client.

PR 98. Ratchet source width at one hundred columns **CODE READ.** Snapshot: draft, open. Public proposal; head 820db0c11d16. **Proposed falsifier:** A newly long non-exempt source line passes, or an unchanged allowed line creates false debt.

PR 99. Qualify a combined descriptor and mapping candidate **CODE READ.** Snapshot: draft, open. Public proposal; head 714ec29d3fdc. **Proposed falsifier:** The exact combined tree fails full-profile Windows or other-die regression, despite parent passes.

PR 101. Inventory stale documentation with a ratchet **CODE READ.** Snapshot: open, non-draft. Public proposal; head b7fd84011a7b. **Proposed falsifier:** A planted stale active document escapes detection or a historical archive is treated as live.

PR 103. Correct two historical status checkpoints **CODE READ.** Snapshot: open, non-draft. Public proposal; head 19dcdd9b3d50. **Proposed falsifier:** The new status still implies the old checkpoint is the current integrated implementation.

PR 107. Calibrate the Windows readiness observer **CODE READ.** Snapshot: draft, open. Public proposal; head 95dac5d5eacb. **Proposed falsifier:** A slower observer merely hides a product failure or accepts readiness without the required state.

PR 108. Measure display fields with a privileged read-only tool **CODE READ.** Snapshot: draft, open. Public proposal; head 2ace3d219ad4. **Proposed falsifier:** The tool alters state or its independently decoded replies disagree with the native oracle.

PR 110. Retire failed provider machines before retrying **CODE READ.** Snapshot: draft, open. Public proposal; head d53cfd9f44d7. **Proposed falsifier:** Retry starts before absence is verified or chooses the same failed machine or host.

PR 112. Consume measured Blackwell display capabilities **CODE READ.** Snapshot: draft, open. Public proposal; head 008c97705a2c. **Proposed falsifier:** Same-die native answers disagree, or the exact-head Windows A/B still fails the target path.

PR 127. Record a Blackwell driver-readiness refusal **CODE READ.** Snapshot: open, non-draft. Public proposal; head f0fa7f4953b9. **Proposed falsifier:** The operator labels a failed provider boot as a tested product cell or fails owned cleanup.

PR 137. Prototype a private controller without the Worker **CODE READ.** Snapshot: draft, open. Public proposal; head 794d4a700d75. **Proposed falsifier:** A real owned-instance lifecycle violates approval, spend, exact-SHA or teardown constraints.

PR 140. Keep retired backing until invalidation acknowledges **CODE READ.** Snapshot: draft, open. Public proposal; head edfd0271f381. **Proposed falsifier:** Old backing is freed while an old translation can execute, or failed invalidate loses ownership.

PR 146. Represent guest sparse pages with selective host reservations **CODE READ.** Snapshot: draft, open. Public proposal; head 3e52e0161c24. **Proposed falsifier:** Sparse-ON repeats the failure under the full production profile or breaks another die.

PR 160. Bring up the TU106 Windows path **CODE READ.** Snapshot: draft, open. Public proposal; head 0b44315ef7f3. **Proposed falsifier:** An exact-head guest fails the full workload despite a successful adapter initialisation.

PR 163. Permit an explicit headless Windows device experiment **CODE READ.** Snapshot: draft, open. Public proposal; head e5b9ee2fe647. **Proposed falsifier:** Display-disabled setup still creates a display device or is mistaken for full desktop acceptance.

PR 164. Model Turing display-window semaphore releases **CODE READ.** Snapshot: draft, open. Public proposal; head c7e085b3d82a. **Proposed falsifier:** The native release oracle and guest notification disagree; acquire semantics remain unmodelled.

PR 165. Reduce repeated Windows lease setup **CODE READ.** Snapshot: draft, open. Public proposal; head a1cd0e6283e6. **Proposed falsifier:** A cached guest differs from the pinned clean image, leaks private state, or fails a cold control.

PR 169. Explain a consumed completion wake **CODE READ.** Snapshot: draft, open. Public proposal; head 6adec88f9a5f. **Proposed falsifier:** An induced lost wake remains invisible, or a harmless pending channel is called stranded.

PR 172. Simplify the GPU trigger workflow **CODE READ.** Snapshot: open, non-draft. Public proposal; head 5ced95e19e8d. **Proposed falsifier:** Removing the action dependencies changes which approved SHA is tested or skips required work.

PR 176. Batch the documentation and site changes **CODE READ.** Snapshot: open, non-draft; head already in baseline. Public proposal; head 12c4008705f1. **Proposed falsifier:** The final batch tree differs from the approved tree or a publication check fails.

6.19 A gap ledger can itself be stale

CODE READ. The feature-gap catalogue mixes closed rows, open questions and corrections. Its priority rule puts application blockers first and console fidelity later. The handoff also retains superseded decisions, including a capability allowlist already approved and a migration alternative later tested natively. Keeping history is valuable only if readers follow dated corrections before reading old verdicts. [gaps] [handoff] [rulings]

INFERRED. The paper has the same failure mode. Its source pin prevents silently moving the implementation under a claim; it cannot prevent a new experiment tomorrow from falsifying an explanation. The inventory should be regenerated for a later edition, and the case study should change only when a discriminating measurement changes it. The unanswered questions that follow are intended to help make that happen.

7 Questions for a reader without the tree

Each setup below specifies a proposed experiment or restates the preceding architecture. Observations are requests, not results. An answer can be a public specification, a counterexample or a controlled experiment; no private trace or insider branch name is needed to understand the fork. The source keys in the preceding sections establish the motivating evidence.

Q1. Must every mapping barrier scan the whole space?

Setup: A stock driver changes a small range in a large GPU VA space, then submits a mapping barrier. The emulator has confirmed host placements but no trusted changed-range announcement.

Observation to obtain: Identify an open-interface certificate that proves which table paths could change, or a counterexample where any bounded local check misses a relevant update.

Decision it changes: A valid certificate justifies an incremental walk. Without it, the whole-space scan remains a correctness cost and optimisation must target apply work or precomputation.

Q2. Can unchanged ranges stay live through an own-space refresh?

Setup: One host address space has running work on range A while the guest remaps disjoint range B. The intended outcome for B during transition is old backing, new backing or fault.

Observation to obtain: Give a bounded validation protocol that lets A publish safely despite root changes, aliasing and an overflowing change log.

Decision it changes: This decides whether per-target holds can become per-range admission or must remain conservative.

Q3. Which event proves that old backing is no longer used?

Setup: An emulator unmaps a host VA, asks the host driver to invalidate and wants to free the old memory object while earlier GPU work may still exist.

Observation to obtain: State the documented ordering guarantee or a counterexample where access continues after the acknowledgement the emulator treats as retirement.

Decision it changes: A weaker guarantee requires a separate lifetime fence; a stronger one can simplify deferred frees.

Q4. Is the shared kernel copy stream the wrong granularity?

Setup: A guest kernel uses one copy channel for work requested by several applications. Each mapping split serialises later work on that channel.

Observation to obtain: Distinguish a driver-required shared order from an implementation-created dependency using per-request completion identities and separate target spaces.

Decision it changes: An inherent shared order calls for cheaper splits; an artificial one calls for changing grouping or admission while preserving the stock protocol.

Q5. Does a raised watchdog reveal lateness or mask another failure?

Setup: The same fixed workload is run with default watchdog settings and a longer diagnostic window. A host publication, a processed marker and guest completion are independently observed.

Observation to obtain: Show whether the same outstanding work eventually completes, with matching life identity, rather than being canceled and replaced during recovery.

Decision it changes: Only the first outcome supports late execution. A replacement or ambiguous identity reopens the missing-completion explanation.

Q6. What clock would separate execution from observation delay?

Setup: A host records when it publishes work and when a CPU first reads its completed marker. The observing worker may block in a host driver call.

Observation to obtain: Find a usable GPU timestamp or independent witness whose ordering can be related to the host clock without adding a correctness poll.

Decision it changes: This decides whether publication-to-marker time can be decomposed or must remain an ambiguous sum.

Q7. Can an interrupt be correlated with a packet at all?

Setup: Host notifications are engine-wide and may arrive before the CPU samples a marker. Several channels and VMs can request the same kind of notification.

Observation to obtain: Specify the minimal identity and ordering observations needed to distinguish a covering signal from an unrelated one under coalescing and masking.

Decision it changes: Without that proof the diagnostic must report incomplete correlation rather than a fabricated marker-to-interrupt latency.

Q8. Can a finite sparse budget preserve the guest contract?

Setup: A guest asks for more sparse reservations than the emulator can afford. A closed fallback leaves missing coverage invalid and counts it.

Observation to obtain: Identify a stock-driver-visible refusal point that is recoverable, or show a workload that requires sparse semantics after that point and cannot recover.

Decision it changes: This chooses between a supported bounded mode, larger representation changes and explicitly unsupported workloads; a silent budget increase alone does not answer it.

Q9. Can sparse geometry be coalesced without a hole?

Setup: Two adjacent sparse reservations are live and unchanged. Replacing them with a larger object would reduce resource cost, but release-before-reserve can temporarily fault them.

Observation to obtain: Find a host operation with atomic replacement semantics, or prove a transition sequence that preserves every unchanged address without pausing the GPU.

Decision it changes: A valid primitive enables compression. Otherwise active sparse geometry cannot be reclaimed like empty ordinary reservations.

Q10. What bounds a single native call?

Setup: Software stops starting optional work after a slice budget, but one native driver operation can outlive that budget and repairs cannot be abandoned.

Observation to obtain: Measure the per-operation tail on loaded and idle hosts and identify any documented cancellation or asynchronous completion interface.

Decision it changes: A real bound enables deadline reasoning. No bound means the product must tolerate the latency or move the dependency ahead of the guest-visible barrier.

Q11. Can optional geometry starve safely?

Setup: Mandatory mapping splits continue indefinitely, leaving no idle opportunity for deferred sparse refinement. Existing accepted backing must stay owned.

Observation to obtain: Determine whether a guest can continue correctly with the counted deficit forever or requires a progress guarantee for the optional work.

Decision it changes: This decides whether idle-only continuation is a complete mode or needs explicit fair admission.

Q12. What can coalescing remove from a serial requester?

Setup: One channel requests a fresh walk only after the previous request completes; other channels sometimes request the same target concurrently.

Observation to obtain: Separate saved concurrent tickets from the irreducible serial request count and measure both.

Decision it changes: This prevents a batching optimisation from being credited with a fix it cannot provide and directs effort to the remaining per-request work.

Q13. Is native ring relocation a simpler stock path?

Setup: A stock Windows driver asks to move a running user channel's ring. The emulator can either mediate from birth or use an authored native relocation with proven ordering.

Observation to obtain: Demonstrate cursor continuity, old-ring retirement and a replacement-only completion marker through a move while work is already in flight.

Decision it changes: Success could retire part of the mediation machinery; failure preserves the need for a restricted software relay or a clearly separate cooperation tier.

Q14. What makes a registration anchor trustworthy?

Setup: A registration reply changes how later command entries must be inspected. The relay has observed PUT at several earlier points and may cache it.

Observation to obtain: Give an ordering proof or counterexample establishing which observed boundary precedes the reply under concurrent guest publication and ring migration.

Decision it changes: The answer determines how much must be scanned and whether the chosen clean anchor is valid.

Q15. Can pointer mode remain independent of command bytes?

Setup: The GPU executes user commands under USER privilege in a restricted VA space. Software still validates each descriptor but Pointer mode promises to read no pushbuffer.

Observation to obtain: Find a software action reachable through uninspected command bytes that escapes that authority boundary, or prove why the hardware boundary suffices for ordinary commands.

Decision it changes: A counterexample narrows Pointer eligibility; a sound boundary avoids copying commands solely because a guest can mutate its own memory.

Q16. What is the smallest useful ring for lazy observation?

Setup: The relay intentionally writes GET back later than hardware progress. A producer can stop when its small ring appears full.

Observation to obtain: Construct the worst ordering of publication, observation, credit checks and processed-marker wake for a ring only a few entries long.

Decision it changes: This decides whether the effective observation interval is a proven bound or a benchmark-shaped heuristic that needs a different rule.

Q17. What must a holder promise before a waiter parks?

Setup: A worker fails a try-lock while an earlier doorbell edge has already been consumed. The lock holder may release just before or just after waiter registration.

Observation to obtain: Supply a bounded register-recheck-release protocol whose final wake cannot disappear under either ordering, including reset and object reuse.

Decision it changes: A counterexample changes the handoff protocol; repeated successful stress runs alone do not remove the missing interleaving.

Q18. Can a progress oracle avoid sharing the implementation's mistake?

Setup: Both an implementation and a reference model represent a wake obligation, and both may have been written from the same informal story.

Observation to obtain: Derive an independent conservation law, such as owner-or-queue-or-published-bit, that can reject a terminal pending state without copying the implementation transitions.

Decision it changes: This selects a stronger model oracle and reduces correlated false green results.

Q19. Is finite fair-suffix liveness the right contract?

Setup: An explicit-state test explores adversarial prefixes and then gives every live actor fair turns after finite lock holds.

Observation to obtain: Identify a realistic host execution excluded by those assumptions that the product still promises to survive.

Decision it changes: The answer changes either the model scheduler or the claimed product progress guarantee; an unexplored frontier must remain inconclusive.

Q20. How much cross-client authority survives one UID per VM?

Setup: Separate VMM processes hold driver clients on one physical GPU. Each runs with a distinct unprivileged UID and a restricted channel address space.

Observation to obtain: Document or test remaining operations that can name another client's resources without its explicit cooperation.

Decision it changes: This decides whether the deployment policy closes the relevant boundary or only one path.

Q21. What is the right live victim for a GPU fault test?

Setup: One guest deliberately faults while another has an existing context, queued work and data that must remain correct across the event.

Observation to obtain: Observe victim completion, content, latency, reset scope and later reuse, with a matched native process control.

Decision it changes: A global failure restricts sharing claims; survival supports only the exercised fault class.

Q22. Which resource needs a quota first?

Setup: Address spaces isolate normal memory accesses, but tenants share scheduling, encoder capacity, host driver calls and memory apertures.

Observation to obtain: Find the cheapest unprivileged workload that prevents a live peer from making useful progress and measure recovery after it stops.

Decision it changes: This prioritises admission and fairness controls by observed impact rather than by ease of counting.

Q23. Can engine-wide interrupt noise become material denial of service?

Setup: Every VM registered on an engine can receive spurious notifications caused by another tenant. Correctness requires retaining real completion debt.

Observation to obtain: Measure victim CPU and completion latency under an interrupt-heavy neighbour, then test pacing that never erases pending work.

Decision it changes: This decides whether the accepted noise needs a bounded-service policy before wider sharing.

Q24. Is a per-die table fundamentally avoidable?

Setup: The guest asks for a capability absent from open derivation and unprivileged host queries. Several measured dies agree, but no mechanism explains the agreement.

Observation to obtain: Provide a public derivation or a new die that distinguishes a family rule from coincidental agreement.

Decision it changes: A derivation deletes a table; otherwise exact-die measurement and a tested fallback remain necessary.

Q25. What should an unmeasured die refuse?

Setup: A virtual device can advertise fewer optional display features while keeping the guest driver unmodified.

Observation to obtain: Identify the smallest capability surface that completes a realistic application and recovery suite on the die, with wrong-answer mutations proving each required feature matters.

Decision it changes: This defines a working fallback mode instead of equating refusal-first with usable compatibility.

Q26. How should the two driver axes be sampled?

Setup: Host and guest driver versions are independent, while interfaces may change inside a release branch. Exhaustive combinations are expensive.

Observation to obtain: Choose a public-interface change criterion that selects discriminating cells and test whether it predicts actual failures better than major-version sampling.

Decision it changes: This changes the supported matrix and which cells need hardware rather than a decoder-only check.

Q27. Does a successful query imply the advertised virtual feature?

Setup: A physical host reports a display capability that the emulated display only partly implements.

Observation to obtain: Mutate each advertised field and compare guest behaviour and output against an independently derived virtual support model.

Decision it changes: This decides whether runtime delegation can replace a table or must be filtered field by field.

Q28. How should a device reboot differ from a driver reset?

Setup: A new guest OS life must not see stale video memory from the previous life; an in-guest driver recovery belongs to the same trust boundary.

Observation to obtain: Show that the VMM reset origin and type distinguish every reachable lifecycle event, including a reboot arriving during another reset.

Decision it changes: A missing distinction requires a conservative scrub path or a better VMM lifecycle contract.

Q29. What proves repeated unload and reload?

Setup: A cold boot works, but a guest repeatedly unloads its driver, reopens it and starts new work without restarting QEMU.

Observation to obtain: Track queue identity, object ownership, host resources, content and completion over many lives and compare a kill-during-work control.

Decision it changes: This decides whether a lifecycle is supported or merely modelled, and which resources need stronger generation identities.

Q30. Why does one OS resume while another does not?

Setup: The firmware-facing protocol can retain state through S3, but Windows also has timer and platform dependencies outside the emulated GPU.

Observation to obtain: Use matched plain-QEMU and device-enabled controls, guest stack evidence and post-wake GPU work to locate the first divergent layer.

Decision it changes: A platform-only failure changes the harness diagnosis; a device-specific failure remains a product blocker despite a healthy firmware state machine.

Q31. What makes arm64 a new architecture test?

Setup: The logic crates can build for another CPU, but the VMM memory adapter must implement mapping, page sizes, DMA visibility and reset semantics.

Observation to obtain: Identify the first required shell operation whose semantics differ, then execute it on a VM-capable ARM host with a real GPU.

Decision it changes: This decides whether portability needs a new adapter capability or only implementation work; cross-compilation alone cannot settle it.

Q32. What is the minimal second-VMM contract?

Setup: The core wants memory views, bounded traps, interrupts and lifecycle events independent of QEMU.

Observation to obtain: Implement a small second adapter and name every place it cannot express the required ordering or ownership without changing the core.

Decision it changes: This tests architectural independence and prices a vfio-user or other frontend without claiming that an abstract trait is already portable behaviour.

Q33. What additional evidence would make multi-GPU real?

Setup: A guest can enumerate and use two distinct GPUs in thin tests, but applications may request peer access, shared managed memory or a display on a chosen device.

Observation to obtain: Run content-checked concurrent application work with explicit peer refusal, device selection and independent reset accounting.

Decision it changes: This separates basic enumeration from a supported multi-GPU product and identifies which cross-device features must stay refused.

Q34. Can display completion be independent of the viewer?

Setup: A guest keeps flipping while the broker is silent, restarts, or never releases a frame slot. The viewer must not control guest liveness.

Observation to obtain: Check monotonic guest completion and bounded frame debt through silence, reconnect and flood without using a timeout to manufacture a GPU completion.

Decision it changes: This decides whether frame transport and guest display timing are properly separated.

Q35. Which visual oracle catches a colour error?

Setup: A pipeline applies matrices and lookup tables with driver-specific widths, rounding and mirroring, while a screenshot can look plausible.

Observation to obtain: Use boundary values and native pixel comparisons that distinguish sign, scale, interpolation and table capacity, including the control that previously froze.

Decision it changes: This chooses behaviour-level tests instead of treating accepted method packets as correct output.

Q36. What ratio survives a nested host?

Setup: A guest under nested KVM pays both an exit cost and emulator work. A same-box host control omits one of those layers.

Observation to obtain: Measure exit count, per-exit latency, batch shape, placement and content on matched nested and physical hosts without changing driver or workload at the same time.

Decision it changes: This separates a host-environment penalty from a product bottleneck and prevents extrapolating one ratio to a different deployment.

Q37. Which sample unit estimates the failure we care about?

Setup: Many iterations in one boot agree while boot state, host allocation and concurrent applications can vary.

Observation to obtain: Separate within-boot, between-boot and between-machine spread, retaining timeouts and failed boots in the denominator.

Decision it changes: This chooses repetitions for a release criterion; a fast median must not discard the unstable lives a user would actually encounter.

Q38. Does an application PASS mean completed GPU work?

Setup: An application exits zero and may print success even when a sync status or numerical result was not checked.

Observation to obtain: Plant a missing launch, wrong result and forced GPU error and show that the independent harness rejects all three while accepting the valid native control.

Decision it changes: This determines which application receipts can grade product correctness rather than startup.

Q39. What can an untrusted GPU gate prove?

Setup: A rented operator receives approved source and a run-scoped reporting capability, but can fabricate output and a structured verdict.

Observation to obtain: State which regression signal remains useful under this trust model and which release claims require an independent trusted builder or observer.

Decision it changes: This keeps the gate a tripwire and prevents it from becoming an accidental provenance authority.

Q40. Can exact-tree batching miss an interaction?

Setup: Several topics each passed a relevant parent revision. They are then merged into one batch against a moving integration branch.

Observation to obtain: Test the final tree, preserve its embedded revision and compare changed input closures to the receipts being carried forward.

Decision it changes: This decides which earlier results remain applicable and when a fresh hardware run is necessary.

Q41. How can a mutation campaign prove it exercised its universe?

Setup: A test runner can time out, omit targets, use stale artifacts or fail before executing the intended mutation.

Observation to obtain: Plant known failures, count reached targets and require an unmutated green baseline plus independent failing assertions rather than compiler errors.

Decision it changes: This changes the interpretation of a kill percentage and prevents an empty or broken campaign from looking stronger than a real one.

Q42. What should an agent reviewer be unable to assume?

Setup: An implementation, its tests and a research summary may repeat one author's untested premise.

Observation to obtain: Require the reviewer to reconstruct one invariant from the opposite boundary and name the observation that would falsify the shared story.

Decision it changes: This changes review assignments and evidence requirements when apparent independent agreement is only repetition.

Q43. Which release artefact is actually distributable?

Setup: The source offers licence alternatives with file-level exceptions, and the combined VMM links components under different terms.

Observation to obtain: Obtain a reviewed source-to-binary licence inventory and verify that each redistributed component has the required permission and notice.

Decision it changes: This decides packaging and release gates. The question asks for legal review, not a technical test result standing in for permission.

Q44. What would make this paper obsolete in the useful way?

Setup: A pinned architecture argument separates measured results from hypotheses, but development continues after publication.

Observation to obtain: Choose one open fork, preregister the discriminator and publish both outcome and instrument reachability at an exact revision.

Decision it changes: A result that closes or changes the fork should update the parent claim visibly. A new fast number without a causal discriminator should not rewrite the explanation.

A Method and reproducible commands

This edition used source reads, Git ancestry checks, the claims ratchet, publication-gate tests and XeLaTeX. It ran no Cargo or Clippy command, VM, real-GPU gate, provider operation or remote host experiment. Every hardware result is carried from the named repository record. The experimental source index is deliberately separate from the implementation baseline.

A.1 Reconstruct the source snapshot

```
git fetch origin next master
git worktree add -b codex/whitepaper-v3-20261027 \
  ../kayfabe-paper origin/next
cd ../kayfabe-paper
git rev-parse HEAD
git log --first-parent --oneline origin/master
# The implementation pin must equal the cover, not a later moving ref:
git show 12c4008705f17033a803e56d4d479bf1275cdb94:AGENTS.md
# Experimental code is inspected without changing the baseline:
git show 2196a9ad1:docs/design/ENS11_SPLIT_LATENCY_20261027.md
```

The worktree command documents how this writing branch was created; it is not a suggestion to reuse an existing branch name. `sources_v3.json` records immutable paths and Git blobs. It can be checked without building the product:

```
python3 - <<'PY'
import json, subprocess
from pathlib import Path
p = Path('docs/whitepaper/sources_v3.json')
for row in json.loads(p.read_text())['sources']:
    ref = row['revision'] + ':' + row['path']
    blob = subprocess.check_output(['git', 'rev-parse', ref], text=True)
    assert blob.strip() == row['blob'], ref
PY
```

A.2 Reconstruct the PR inventory

```
gh pr list --state open --limit 200 \
  --json number,title,headRefOid,baseRefName,isDraft,url
# For a specific snapshot head:
git merge-base --is-ancestor <head> \
  12c4008705f17033a803e56d4d479bf1275cdb94
```

The live query will differ after publication. `pr_snapshot_v3.json` preserves the captured numbers, immutable heads, draft state and the paper's proposed falsifiers. It does not archive mutable PR bodies or claim that the proposals were tested here. The inventory is a catalogue of research and integration debt, not a merge recommendation.

A.3 Build and verify the publication

```
python3 -m unittest discover -s docs/whitepaper -p test_build_v3.py
```

```
python3 scripts/ci/debt.py claims
git diff --check
# Commit TeX inputs before producing a publishable PDF.
bash docs/whitepaper/build.sh v3
python3 docs/whitepaper/build_v3.py --check
pdfinfo docs/whitepaper/kayfabe_v3.pdf
pdftotext -layout docs/whitepaper/kayfabe_v3.pdf -
```

CODE READ. The v3 target runs XeLaTeX exactly twice with halt-on-error and an input recorder. The final log must have no TeX error, missing glyph, undefined reference or unresolved citation. Repository inputs must match committed Git blobs before export. A JSON receipt binds the input paths and hashes, source commit, compiler version and output digest. A later PDF-only commit leaves the input blobs unchanged; the receipt remains checkable. This is a local publication check, not a signed proof of a trusted compiler or host.

A.4 How a hardware result should be reproduced

These are repository entry points, listed as a method rather than commands run for this paper. Use an authorised, exclusively leased machine and record its real configuration first. The gate and thin-suite commands come from the contributor verification rules. [rules]

```
bash scripts/bench/v3_gates.sh
KF_DEVICE=kf3 bash scripts/fastguest/fast_suite.sh <tag> 180
bash scripts/bench/cuda_ladder.sh host <tag>
bash scripts/bench/cuda_ladder.sh guest <tag>
bash scripts/bench/perf_compare.sh <rev> \
  --baseline <rev-or-result-dir> --lanes bare,fast --reps 3
```

CODE READ. The performance harness documents its own baseline binary, common guest client, quantisation, noise observations and summary commands. A table must preserve revision, die, host/guest drivers, nested status, placement, workload settings, flags, sample unit, all failed runs and the control's spread. A new edition should rerun the chosen cell or retain its dated status; changing the source pin does not update the hardware receipt. [linuxperf]

For the stall case, preregister whether the outcome is application completion, no recovery, latency or a particular causal stage. Preserve both the default-timeout arm and any longer observation arm. A missing native GPU clock, incomplete packet association, capped diagnostic ring or missing guest event must remain a missing observation. Never silently remove it from the denominator. These requirements follow the instrument limits in the experimental note. [latency]

B Glossary of terms and historical names

ABI Application binary interface: layouts and calling conventions. Host and guest interfaces are selected independently.

BAR0 / BAR1 / BAR2
PCI address windows. In this design BAR0 writes publish register demand; the other apertures expose memory without per-access trapping.

CE / GR
Copy engine / graphics-compute engine vocabulary used in source records.

DMA
Direct memory access by the device. DMA progress is not automatically a CPU event.

Doorbell
A notification to inspect already published work; the token selects a route.

GPFIFO / pushbuffer
A ring of command references / the method packets they point to.

GET / PUT
Consumer fetch progress / producer publication boundary. GET is not a general execution-completion fence.

IOVA / GPA
Device-visible I/O virtual address / guest physical address. A virtual IOMMU can make these differ.

IRQ / MSI / DPC
Interrupt request / message-signalled interrupt / Windows deferred interrupt work. A host notification, physical line signal and guest handler are separate events.

MEM_OP / split
A memory-operation method / a software ordering boundary that waits for the required mapping refresh before later work proceeds.

MMU / PTE / PDE / PDB
Memory management unit / page-table entry / page-directory entry / page-directory base. A VA-space object can outlive a particular root.

Non-stall interrupt
An asynchronous engine notification; the observer checks its own completion value rather than treating the interrupt as a per-packet receipt.

RM / GSP
Resource manager / GPU System Processor. Kayfabe models the guest-facing firmware peer and separately uses the real host resource manager.

TSG
Timeslice group: related channel scheduling membership.

USER / USERD
GPU user privilege / channel producer-consumer state memory. USER does not describe the host launcher's Unix UID.

UVM
Unified virtual memory. Fault delivery and replay are separate from mirroring already present mappings.

WDDM / TDR
Windows Display Driver Model / timeout detection and recovery.

Xid / event 153 / Code 43
Host NVIDIA error record / guest driver event used by the campaign / Windows adapter status. These are different observation domains, not synonyms for one root cause.

B.1 Names encountered in historical files

These labels are decoding aids only. The main argument uses functions and revisions instead of asking the reader to remember campaign names.

v3 / pre-v3
Current `kf-*` product tree / frozen `kayfabe-*` predecessor and grader dependency closure. [architecture]

M / B-prime / baton
Names for relay-handoff alternatives and mixed-workload progress experiments. The baseline README cites M's `89fc8a222` result. No broad Windows comparison of every alternative is asserted here. [readme] [alternativerelay]

ENS5 / ENS6 / ENS7
Local sparse-admission undo; RM churn and soft slices; framebuffer-scaled sparse budgets. Their source index points to an experimental revision, not the baseline. [admission] [rmcost] [budget]

ENS8 / ENS10 / ENS11
Consumed-ring wake tests; host-target scope narrowing; split-latency/optional-work continuation integration. CPU validation and hardware acceptance remain distinct. [owed] [walknarrow] [latency]

T-space
Kayfabe's privileged translated-work address space. A guest USER channel must not inherit its whole-memory window. [rulings]

Candidate numbers, lane names and short boot labels
Coordination names, not product versions or evidence grades. An exact source SHA and run record are required to interpret them.

ogkm
NVIDIA's open GPU kernel modules, used as pinned reference sources. A version tag is part of a layout citation; paths and meanings cannot silently move across tags. [credits]

C Evidence index

Keys identify the revision actually read, not a promise that the corresponding result was rerun here. Experimental sources below are outside the implementation baseline. A source may report a host trace that is not committed; the case study explicitly marks that limit.

architecture: Functional map and stated invariants
Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`ARCHITECTURE.md`
rules: Contributor rules and next/master merge policy
Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`AGENTS.md`
debt: Attribution scanner and its stated limitations
Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`scripts/claim_ledger.py`
readme: Dated product and workload summary
Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`README.md`
faq: Comparisons, sharing limits and interrupt boundary
Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`docs/FAQ.md`
credits: Verified positioning and licence notices
Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`docs/howto/credits.md`
notices: Vendor software and trademark notices
Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`docs/NOTICES.md`
license: Repository licensing alternatives and exceptions

Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
LICENSE
security: Threat model, deployment assumptions and dated gaps
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/howto/security.md
windows: Stock Windows setup and registry configuration
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/WINDOWS_GUEST_SETUP.md
requirements: Recorded hardware and driver cells; explicit gaps
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/howto/requirements.md
design: Architecture argument and seven compatibility axes
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/THE_ARCHITECTURE_v3.md
rulings: Dated owner decisions and corrected interpretations
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/OWNER_RULINGS.md
handoff: Historical checkpoints and later updates
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/STATUS_AND_HANDOFF.md
status: Scoped evidence catalogue, with staleness warning
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/STATUS_DETAIL.md
old: Predecessor: method and correction history
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/whitepaper/kayfabe_architecture.tex
translated: Address rewriting and real GPU execution
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/THE_TRANSLATED_PLANE.md
kinds: Privilege, address and completion by channel kind
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/the_three_channel_kinds.md
gspcode: Firmware-facing protocol vocabulary
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-gsp/src/lib.rs
rmcode: Guest resource-manager model
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-rm/src/lib.rs
corecode: Composition of trap and worker planes
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-core/src/lib.rs
trapcode: Bounded trap and token interface
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-trap/src/lib.rs
hostcode: Authored host interface
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-host/src/lib.rs
memcode: VA manager request and result protocol
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-mem/src/vasmgr.rs
diffcode: Diff and acknowledgement model
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.

crates/kf-cuda/src/diffmodel.rs
applycode: Applying changed runs and invalidation
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-mem/src/apply.rs
tsgcode: Host group and member construction
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-host/src/channel.rs
latejoin: Late-member state and scheduling dependency
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_LATE_TSG_JOINER.md
relaydesign: Windows user-work relay design
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_WINDOWS_USER_RING_RELAY.md
lifecycle: Corrected lifecycle and lazy-observation rules
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_USER_RING_LIFECYCLE_20261016.md
eventloop: GET versus processed marker correction
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_USER_RING_EVENT_LOOP_20261014.md
lazycode: Cache and forced-observation predicates
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-chan/src/user_lazy.rs
turncode: Actual relay turn and mode transitions
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
crates/kf-chan/src/user_turn.rs
display: Emulated display and boot-layer policy
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_DISPLAY.md
planes: Pixel and completion oracle
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_DISPLAY_PLANES_ORACLE.md
pacing: Broker credits, clock fallback and test limits
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_DISPLAY_PACING_20261017.md
video: Engine grouping and recorded video correctness
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_VIDEO_ENGINES.md
uvmdesign: Fault-delivery research and stock limitations
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_UVM_DEMAND_PAGING.md
driverdesign: Independent host and guest driver matrix
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_DRIVER_MATRIX.md
constants: Tier ladder, source separation and coverage gaps
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_CONSTANT_STRATEGY.md
provenance: Public provenance policy only; no analysis findings
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_PROVENANCE.md
flags: Diagnostic versus product-mode debt
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_FLAG_INVENTORY.md

cooperation: Stock path versus optional cooperation proposals
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_COOPERATIVE_TIERS.md
gaps: Dated priority and application-gap catalogue
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_FEATURE_GAPS.md
reset: Reset stages and lifetime contract
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_DEVICE_RESET.md
sleep: Linux evidence and Windows resume limits
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_S3_RESUME.md
migrationdecision: Mediation, native relocation correction and cooperation alternatives
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_WINDOWS_USER_FIFO_MIGRATION_DECISION_20261011.md
owed: Consumed-wake witness, model limits and later fix
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_ENS8_OWED_RING_TESTS.md
broker: Broker lifecycle, current policy and unattested verdicts
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
ci/vast-broker/README.md
threat: Owner approval and test-infrastructure trust boundary
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
ci/vast-broker/docs/THREAT_MODEL.md
apps: Application outputs, samples and failures
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_APP_MATRIX.md
winmatrix: Windows application receipts at a named revision
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
traces/win11_userring_20261014/matrix-eb9f2aa3-20261014-2321.md
physevidence: Physical/virtual operand hardware boundary
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
traces/phys_operand_oracle_20261008/README.md
sweep: Exact GA104 driver/gate cell
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
traces/sw3_die_sweep_20261024/README.md
turing: TU116 Linux campaign and its exact checkouts
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
traces/v3_turing_master/README.md
families: AD104 workstation and GB205 Linux campaigns
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
traces/v3_families_master/README.md
linuxperf: Baseline selection, timing methods and limits
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_LINUX_PERF_BASELINE_6692e621.md
winperf: Windows experimental and production timing scopes
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
traces/perf_status_20261017/README.md
llmmethod: Language-model campaign revisions and test method
Revision 12c4008705f17033a803e56d4d479bf1275cdb94.
docs/design/V3_BUILD.md
llmdata: Three-run rates, ranges and text-equality records

Revision `12c4008705f17033a803e56d4d479bf1275cdb94`.
`traces/llm_parity/vh_amd/summary.md`
alternativerelay: Retry-set relay proposal, semantic changes and unrun hardware obligations
Revision `c452ca7cb412852a3e6ca68d18e94f92ac625599`.
`docs/design/V3_RELAY_SIMPLE_20261020.md`
admission: Local undo, occupancy and software churn evidence
Revision `2196a9ad1103052dcc4cd1ce2bf53529b574f030`.
`docs/design/V3_ENS5_SPARSE_ADMISSION.md`
rmcost: Native reservation churn and first-reset counterexample
Revision `2196a9ad1103052dcc4cd1ce2bf53529b574f030`.
`docs/design/V3_ENS6_RM_COST.md`
budget: Finite budgets, missing-coverage accounting and exclusions
Revision `2196a9ad1103052dcc4cd1ce2bf53529b574f030`.
`docs/design/V3_ENS7_SPARSE_BUDGET.md`
sparse: Default-OFF sparse representation and refresh contract
Revision `2196a9ad1103052dcc4cd1ce2bf53529b574f030`.
`docs/design/V3_SPARSE_TWIN.md`
walkscope: Earlier scope proposal and explicit unresolved contracts
Revision `2196a9ad1103052dcc4cd1ce2bf53529b574f030`.
`docs/design/V3_WALK_GATE_SCOPE_20261026.md`
walknarrow: Target-scoping proof limits and supplied boot timing
Revision `2196a9ad1103052dcc4cd1ce2bf53529b574f030`.
`docs/design/V3_WALK_GATE_NARROWING.md`
latency: Split freshness, optional continuation and native timing limits
Revision `2196a9ad1103052dcc4cd1ce2bf53529b574f030`.
`docs/design/ENS11_SPLIT_LATENCY_20261027.md`
retirementpr: PR 140, writing-start snapshot; head edfd0271f381. The PR body is a secondary report; no hardware acceptance is inferred from its open state.
sparsepr: PR 146, writing-start snapshot; head 3e52e0161c24. The PR body is a secondary report; no hardware acceptance is inferred from its open state.
probepr: PR 169, writing-start snapshot; head 6adec88f9a5f. The PR body is a secondary report; no hardware acceptance is inferred from its open state.
indexpr: PR 80, writing-start snapshot; head 5532cdc88ba8. The PR body is a secondary report; no hardware acceptance is inferred from its open state.

C.1 Historical gap names: a coverage index, not current verdicts

The following preserves every named row in the dated feature-gap catalogue, including application-specific rows and entries since closed or superseded. It is a topic index: none is asserted to remain an unfixed defect at the baseline. Read the correction above the source row and the corresponding PR/acceptance record before assigning present status. [gaps]

Historical ID	Question or workload topic
G01	Overlay-plane (MPO) flip completion
G02	MPO policy: advertise no overlay planes
G03	Video playback TDR in the hold (about every 5 min)
G04	HW decode must work or fall back cleanly
G05	Display halt on a refused GPU-side colour program
G06	Publish-then-raise: window event bits not owed
G07	Vidmem display writes may return before the DMA lands
G08	CE notifiers 12/23/24/26 never delivered

Historical ID	Question or workload topic
G09	Invalidate latency (section AD, 12 ms)
G10	Completion-audit findings not individually closed
G11	Batched-map decisions: hardware verdict pending
G12	Pending: zero-flag 15-minute run and 60-minute soak
G13	Pending: Linux regression on the combined line
G14	Translated-CE / kernel work refusals that apps hit
G15	Windows app matrix in progress; full success pending
APP-kf_glgears	Windows OpenGL gears crash; candidate passes
APP-blackscholes_t	Windows BlackScholes failure / startup timeout
APP-clpeak_cuda	Windows CUDA clpeak context-switch timeout
APP-glgears_wgl	Gears timeout after CUDA timeout on slicing candidate
APP-vectorAdd_demo	Short CUDA sample missed by periodic GPU-activity proof
APP-stream_perthread_t	Cold unrelated Torch startup consumes CuPy stream budget
APP-cup8_ladder	Short exact CUDA matmul missed by activity sampling
APP-cupy_sha256	CuPy SHA256 header discovery failure
APP-cupy_cg	CuPy cooperative-groups header discovery failure
APP-managed_t	Windows managed-memory allocation stops at translated SYNC_WAIT
APP-cdp_quicksort	New dynamic-parallelism test cannot load NVRTC builtins
APP-nvenc_av1	NVENC AV1 encodes but triggers recovery
APP-nvenc_h264	NVENC H264 encodes but triggers recovery
APP-nvenc_hevc	NVENC HEVC encodes but triggers recovery
APP-cupy_asynccopy	CuPy asynchronous-copy header discovery failure
APP-minecraft_manual	Owner reports startup freeze on manual1380; not a matrix verdict
APP-gravitymark_vk	Vulkan fails on kayfabe; same-GPU VFIO PASS
APP-furmark_gl_bench	FurMark OpenGL intermittent PBDMA DEVICE failure
APP-oceanFFT_demo	OceanFFT GUI misses per-process GPU proof
APP-randomFog_demo	RandomFog GUI misses per-process GPU proof
APP-cupy_check	CuPy shared correctness script missing CUDA headers
APP-nvdec_h264	Decode completes with guest recovery events
APP-d3d11va_h264	Decode completes with guest recovery events
APP-nbody_demo_gl	N-body GUI fails on kayfabe despite stock GUI PASS
APP-scale_cuda_nvenc	CUDA scaling/encode repeats NVENC session refusal
APP-vulkan_video_decode	Video workload passes with GPU recovery
APP-edge_video_h264	Video workload passes with GPU recovery
APP-edge_video_vp9	Video workload passes with GPU recovery
APP-heaven_d3d11	Heaven window survives but GPU proof is absent
APP-heaven_opengl	Heaven OpenGL exits early with GPU recovery
APP-valley_d3d11	Valley D3D11 wedges the guest supervisor
APP-valley_opengl	Valley OpenGL times out with GPU fault
APP-vkcube	Vulkan cube renders but fails activity proof
APP-furmark_gl_stress	CLOSED: fresh renderer log supplies existing proof
APP-gravitymark_d3d12_raster	Raster run exits without GPU activity proof
APP-gravitymark_d3d12_rt	Ray tracing run fails with driver recovery
APP-torch_ai_bench	PyTorch AI benchmark reboots Windows after a walk capacity refusal
APP-gravitymark_d3d12	D3D12 benchmark fails with recovery
APP-gravitymark_gl	GravityMark OpenGL exits with GPU fault and recovery
APP-gravitymark_d3d11	D3D11 exits without GPU activity proof
APP-clpeak_ocl	OpenCL peak crashes with recovery and temporary-file error

Historical ID	Question or workload topic
G16	ANSWERED 2026-10-14 (V3_USER_RING_SCAN_RACE.md, fixes c45ac776 SyncGate + 50068268 Poll): not a scanner desync — the relay read pushbuffers behind an unreached SYNC_WAIT (the guest fills them later; 15/15 direct refusals, 0 stale reads with the gate) and stopped reading GP_PUT without a doorbell after its ring drained (30 IDLE_STALLs, 1.6-55 s). The text below is the 2026-10-14 01:00 row. Windows USER-ring scanner refusals precede the Win11 resets
G17	2026-10-14 (V3_USER_RING_SCAN_RACE.md §7): kept refused — serving it needs either the host's own engine-resetting verb on a shared GPU or an NV_OK kayfabe did not perform, with the KMD's post-reset expectations not public; both causes that made the guest ask for it are fixed (G16) and it was not reached in the validation lanes. The row below is unchanged; the decision is still the owner's. NV906F_CTRL_CMD_RESET_CHANNEL refused
H01	Captured display caps page
H02	KF3_GFX_POOL_PROBE invented sizes
H03	KF3_DISPLAY_CTRL_PROBE 0x7302a3 echo
H04	Class-B flags still to hardwire (D4)
H05	Refused/unserviced controls that guests repeat
H06	Refusal audit coverage
H07	Class-B refusals left open
H08	Per-VM RUSD and utilization.gpu
H09	EVICT_CTX inference
H10	0x2081010d stub
H11	Per-VM GPU UUID
H12	Guest driver version coverage
H13	Interrupt: GSP queue Release fence
H14	Interrupt: GP_GET refresh before non-stall raises
H15	Reboot, FLR, GSP life cycle
H16	Driver unload/reload in the guest
H17	Multi-head and hot-plug on Windows
H18	Windows Secure Boot and boot-ROM signing at product level
H19	Security audit before the first public binary
H20	Multi-GPU in one VM on Windows
H21	Shared BAR1 budget across VMs
H22	Family breadth
H23	Video lane: RUN_CAP wall without persistence mode
H24	Windows video engines NVENC/NVDEC/OFA
H25	Managed memory (UVM)
H26	Perf regression gate
H27	Host-only tooling not in git
H28	Colour: chroma/TMO decisions
H29	Colour pipeline as default, not a flag
L01	Console YUV composition fidelity
L02	Overlay depth, blend and clip on the console
L03	Scaling filter quality
L04	HDR and 10-bit video on the console
L05	GPU-resident console path
L06	Console-only things Linux already simplifies
L07	Power/thermal/P-state queries
L08	Suspend/resume, hibernate, Modern Standby
L09	Host patch tier
L10	Peer-to-peer inside a guest
L11	Mixed-family in one process
L12	NVJPG, SEC2
L13	NVENC session cap accounting, 0x20808165
L14	Perf work on the display and VA paths

D What changed from the predecessor

CODE READ. The old text describes the earlier multi-process/isolate architecture and records corrections through its September restamp. It remains an archive. This edition starts from the single-process v3 product, a GPU-driven page-table diff, the current channel kinds and the Windows user-ring lifecycle. It does not carry forward old line counts, mutation percentages, performance floors or claims that a feature has never run. [old] [architecture]

The audience is broader: each driver-internal concept is introduced before the architecture. The uncomfortable section separates the older wake defects from the later sparse experiment, records where a proposed bound was false, and carries a complete open-PR snapshot instead of implying experiments have landed. The outside-reader questions state setup, requested observation and the decision it changes. The publication gate records committed inputs and checks the finished log, because producing a plausible PDF is not evidence that TeX succeeded.

The source pin is explicit and immutable. Experimental sources have a second revision and reported observations retain their missing raw-trace limitation. This is the successor's method, not a claim that the old measurements were all wrong or that the new explanation cannot fail in the same way.